

The Operator's Manual

*How to run, configure and operate the BSS — by role and by task. The companion volume, *The Honest Machine* (a build memoir · verify everything — how one person and an AI built a complete telecom suite and proved every piece of it), tells how it was built and why; this book tells you which button, which endpoint, which environment variable. Everything here is proven by the suite named beside it.*

Contents

1. Getting started
2. Surfaces & sign-ins
3. The product owner
4. The CSR
5. Billing & money operations
6. Partner channels
7. The host administrator
8. AI configuration
9. The seam catalog
10. Environment reference
11. Extension API reference
12. The mock integrations
13. Production operations
14. Privacy & GDPR operations
15. The agent channel (agentic commerce)
16. The digital workforce
17. The wrapped legacy estate (overlay)
18. Generative discoverability (GEO)
19. The capability map
20. Loyalty — reward & retain
21. Revenue export — the subledger
22. Credit notes
23. Process flows & the incident agent
24. The standards sweep
25. The product configurator
26. Service qualification & the coverage map
27. AI management — the control plane's face
28. Risk management — the door check
29. Partnership types & geographic sites
30. The proof run & fleet-age runbooks
31. The shop that knows you
32. The CTK campaign — twenty-five kits, the closed list
33. Bring your own CMS/DAM — reference-mode content
34. Bring your own carrier — the delivery menu
35. Bring your own PSP — cards, Klarna/Vipps/PayPal, BNPL books
36. The honest game — referrals, streaks, right-plan, community
36. Bundle decomposition — the several things it is
37. Wholesale / open-access — selling on a wire you don't own
38. Browse-path caching — the shop on a surge
39. The fixed wholesale console — the owner's authoring desk
40. Mobile wholesale / MVNE — usage-metered settlement
41. Extending the provider seams — bind, wire, or build
42. Collections — the statutory ladder
43. Device commerce — financing, trade-in, withdrawal
44. Base migration — moving the installed base
45. Usage policy — pools, caps & the roaming wall
46. Norway rails — registry, directory, credit, bill channels
47. Real-time charging — a real OCS behind the seam
48. Device entitlements — what the phone may use (GSMA TS.43)
49. The ontology — what the BSS can do, written once

1 • Getting started

```
docker compose up -d          # ~90 containers; first build takes a while
open http://localhost:8080/shop/ # the storefront (guest browsing works)
```

The gateway is :8080 ; Keycloak is :8085 (admin/admin). Two seed operators run side by side: **genalpha** (EN/EUR, realm `bss`) and **nova** (NO/NOK, realm `nova`) — plus any operator you mint (§7). Demo card 4242 4242 4242 4242 pays; anything ending 0002 declines. Promo code WELCOME10 . Serviceable fiber postcodes start 111 / 222 / 333 .

The one discipline: every feature in this manual has an end-to-end Playwright suite in `ops/e2e/` . When in doubt about how something behaves, the suite is the specification: `node ops/e2e/<name>_test.js` runs it against the live stack.

2 • Surfaces & sign-ins

URL	WHO	DEMO SIGN-IN
<code>/shop/</code>	Customers (self-register or guest)	<code>kai@bss.local</code> / <code>kai</code>
<code>/app/</code>	Customers on mobile (Expo web)	<code>emil@acme.example</code> / <code>emil</code>
<code>/biz/</code>	B2B org admins + invited members	<code>bianca@acme.example</code> / <code>bianca</code>
<code>/csr/</code>	Agents (role-scoped powers)	<code>agent-anna</code> / <code>agent ; junior: jo@bss.local</code> / <code>jo</code>
<code>/console/</code>	Back office (role-scoped tabs)	<code>demo</code> / <code>demo ; product-only: pat@bss.local</code> / <code>pat</code>
<code>/dealer-app/</code>	Retail-partner clerks & telesales agents	any account whose org holds a dealer agreement
<code>shop.nova.localhost:8080/shop/</code>	Nova's white-label storefront	<code>nils@nova.local</code> / <code>nils</code>
<code>biz.nova.localhost:8080/biz/</code>	Nova's B2B console (Norwegian)	<code>birgit@fjellheim.no</code> / <code>birgit</code>

White-labeling is hostname-driven: `shop.<tenant>.localhost` serves that operator's brand, locale and currency from the live registry (§7).

3 • The product owner

Catalog (console → Product Offerings / Specifications / Prices)

Offerings are TMF620 data: hard and soft bundles (`bundledProductOfferingOption` cardinality, pick-N-of-M choice groups), characteristic-conditioned prices (" +2.00/mo when colour = Titanium"), commitment terms, categories that drive placement *and* fulfilment (Top-ups and Insurance are billing-only; Partner services mint entitlement codes; everything else draws an MSISDN + SIM). Artwork rides the offering's attachments — internal store or the tenant's own PIM; the internal store's bytes live in-row by default or in S3-protocol / Azure Blob object storage (deployment config, §9 — suite #54 proves all three).

Catalog lifecycle — launch is a decision, not a side effect

The TMF620 ladder has server-side TEETH: non-staff callers only ever see offerings that are **Launched/Active and inside their validFor window** — the list filter is forced, a draft is 404 by id, and ordering one (including a bundle child) is refused at validation. Per-tenant switch catalog-governance (operator form / console Operators pane): `direct` (default) keeps create-is-live; `governed` lands every create at "In design" and enforces the ladder *one rung at a time* (In study → In design → In test → Launched → Retired → Obsolete; pre-launch states shuffle freely). `validFor` on the offering is the shelf: before the window it is invisible and unorderable, after it it is gone — enforced at query time, no tick decides visibility. When a Launched offering's window opens, `ProductOfferingLaunchedEvent` fires exactly once for launch-day journeys. Staff walk the unlaunched shelf in the real storefront with `?preview=1` — a PREVIEW badge on everything not yet live. Suite `lifecycle_test` proves the whole ladder on a throwaway operator.

Rules & pricing as data (console → Rules)

Order rules and dynamic pricing are JSON-logic rows with a dry-run panel. A price change is a rule, not a release. See `docs/product-rules.md` for the authoring guide.

Product Copilot (console → Copilot)

Chat a product into the catalog: the model proposes specs/prices/offerings as TMF620 payloads, the console shows a human card, and a deterministic executor applies it with *your* token on confirmation. The model never writes. The copilot also authors **pricing rules** ("10% off the phone with this plan") and — suite #61 — **personalization experience rules** ("device browsers see this banner and this pin"): both become policy rows you can disable in Rules. **Voice input:** the  button uses the browser's speech recognizer (Chrome/Edge; hidden where unsupported) — the transcript lands in the input and the human still presses Send; the mic never auto-sends. Server-side STT (bring-your-own Whisper) is a named seam for later.

Product Advisor (console → Product advisor) — suite #52

Findings with receipts: top-up attach counted from inventory ("2 of 2 customers on this plan also bought top-ups"), market price gaps from the tenant's market feed (10% materiality threshold — small gaps are deliberately silent), an LLM narrative that never invents a number. **Adopt as draft...** births an `In study` offering; nothing reaches the shop until you promote it.

Campaigns & growth (console → Campaigns / Journeys / Audiences)

Segments form from consented shop behavior; campaigns prove lift against real holdouts; journeys read the customer before speaking; A/B arms split deterministically; guardrails (frequency caps, quiet hours) are settings. Every message lands on the TMF683 timeline.

4 • The CSR

Search the 360 (multi-word, name-ranked, with a trigram typo net — "Solvieg" finds Solveig when strict matching finds nothing), then act — every action logs itself on the customer's timeline:

THE CUSTOMER ASKS	THE BUTTON / ACT
"What's my PUK?" / "reset my PIN"	Reveal PUK · Reset PIN (owner notified — a silent credential change reads as fraud)
"I lost my SIM"	Replace SIM (old card quarantined first)
"new number, please"	Change number (old number quarantined with its story)
"pause my subscription"	Pause / Resume (charging pauses at the OCS; auto-resumes on the promised date)
"I can't pay all of it"	Installments (parts sum exactly; miss one → reminder → grace → dunning)
"this charge is wrong"	Dispute (collection pauses; resolution credits or upholds, in writing)
"send me my invoice"	PDF (the exact document) · Email bill (to the address on file, never one dictated on the phone)
"my internet is slow"	Diagnose (triage across usage/assurance/network facts)
"give my number to my colleague"	Transfer (three-way authorization; B2B handover keeps the company paying)

Self-care mirrors most of these in the shop and app — customers act on their OWN line (ownership-checked; a foreign line answers 404, never 403).

The workspace — who is this, what is happening, what next

The customer page is a **cockpit**, not a filing cabinet. Under the identity line come three small cards — open tickets, orders in flight, the last contact — then the lines and their health, then money (usage, agreements, bills, promotions and cards), then orders and visits, then the timeline. Sections with nothing in them fold into one line of chips ("Nothing on file: spend policies · household pool · auto top-up off"); a section that holds a warning — a barred line, an open dispute, a paused service — is always open. The tertiary line actions (transfer, change number, reset PIN) sit behind *More...*; **Cease** never hides, because a control you cannot see is a control you cannot judge.

The chrome is one bar under 60 px, and every network incident is **one line** — count, the newest names, services affected, since when — with details on click and a dismiss that lasts the session (a new incident brings it back). Whether an incident touches *this* customer is said on the customer's page, not in the banner.

GenAlpha Assist sits beside every customer and is grounded or it is nothing. Its *situation* (an open problem on one of this customer's services, a paused line, an open bill) and its *recommended action* come from the operational ontology (`GET /ontology/v1/context/customer/{id}/recommendations`): a governed action such as resuming a paused line or moving a plan up within its family is dry-run first and shows its conditions, permission and policy answer under *Why*; **Do it** executes it with the agent's own rights and logs the interaction; **Not relevant** is recorded for desk learning. Below it the copilot's summary and an *offer to consider* (on request, never auto-run), and the knowledge the situation asks for — preview inline or **Send** it to the customer's inbox, logged.

The search box takes anything the caller reads out — a name, an email, a phone number, or an order / ticket / customer / product id — and lands on the customer, saying how it got there; the six customers opened last are chips above the results. The ticket queue is a dense list (severity, issue, customer name, age, time in state, status) with the selected ticket worked in a panel beside it. Suite #126; design notes in `docs/csr-workspace.md`.

The knowledge base at the desk

Articles are audience-shelved (customer / CSR / sales / product owner) and the CSR's ask-with-sources grounds its answers in them. Search is full-text, ranked, and stems in **the tenant's own language** — the locale that picks the storefront's currency also picks the stemmer, so on nova "regning" finds "regningene" (suite #55). Typos in customer search are caught by a trigram net that only speaks when strict matching finds nothing — strict results are byte-identical to before. Beneath the keyword search sits the **semantic net** (pgvector): "why is my internet so slow" finds the fair-use article that contains neither word, and nonsense finds nothing — the cosine ceiling keeps the net honest. Embeddings are a seam: `bss.knowledge.embeddings = stub` (deterministic, keyless) or `openai-compatible` (any real model).

5 • Billing & money operations

The billing run

`POST /tmf-api/customerBillManagement/v4/billingRun` (staff). Segment-based and per-account: mid-month starts, mid-cycle plan changes and ceases each bill exactly their own days, labelled ("from the 16th, 15 days"). Payday alignment per customer (`POST /individual/{id}/billingCycle`, day 1–28). The invariant the suites enforce from every direction: **no day is ever billed twice**.

The migration rehearsal — the parallel run before anyone moves

`POST /migrationRehearsal` (billing, `billing:admin`) takes a legacy export as rows `{externalRef, offeringName, expectedMonthly}` and prices each against THIS catalog with the **same engine that cuts real bills**: matched within tolerance, price-differs by exactly how much, or offering-missing — the exceptions BY NAME, an honest `readyToCutOver` flag that is only true when the exception lists are empty, and a read-only promise in the assumptions (the persisted report is the only write). The scariest sentence in a BSS sale — "we'll migrate you" — becomes a report. Suite `migration_rehearsal_test`.

The shadow-operator clone — simulate by running the real thing

`POST /onboarding/v1/operator/{sourceId}/clone {id}` (host desk) mints a **sandbox operator that IS the source, again**: same form-onboarding machinery, the source's catalog and rules copied over the tenants' own staff tokens (ids remapped), and `sandbox: "true"` stamped in its fleet block — attested in the gateway manifest so no channel can mistake it for production. The **wall**: a sandbox tenant can never reach the outside world — the communication egress (email/SMS/push, prospect raw-email included) stores the message in-app marked `sandbox-suppressed`, never delivered. The **answer**: `GET /portfolioDiff?tenantA&tenantB` prices both shelves with the same engine that cuts real bills — matched offerings with per-name deltas (a price move ripples through every bundle that carries it), only-in lists, portfolio totals, assumptions on its face. Mutate the clone, read the diff, throw the clone away: simulation with zero model drift. The clone has since grown the full simulation substrate: **Tvilling base seeding** (`seedTwinBase` — the only thing read from the source is the aggregate offering distribution; the clone mints proportional synthetic subscribers, fictional by construction, sandbox-only by guard), the wall extended to **every door** (PSP sessions and card charges refused, ad-platform activations refused, the GA4 seam silent in both directions), **wholesale rate cards** travelling with the copy, and **time compression**: a per-tenant clock seam (`clock-offset-days`, `sandbox-gated` structurally) that billing periods, dunning's overdue decisions and usage's wholesale windows all read — `simulateQuarter` bills three compressed months in minutes, no day billed twice. Suite `shadow_clone_test`, twelve legs.

The commercial simulator, complete — five answers off the real engines

Prospect simulator (`POST /onboarding/v1/prospectSimulation`): a prospect's PUBLIC price list + an assumed mix → a fresh walled sandbox, their shelf, a twin base, a billed compressed quarter — "your business on our BSS" in the first meeting, touching none of their data. **Chaos twin** (`GET /chaosReport?scenario=psp-outage&days=N`): a failure mode priced in currency off the real ledger — revenue at risk, bills hit, how many age beyond terms before the outage ends; clock-composable in a clone. **Regulatory rehearsal** (`GET /ai/v1/priceRiseRehearsal`): the right-to-exit window counted before any letter exists — the cohort IS the notification list, port-out exposure = churn-flagged holders, upside vs annual-at-risk, with the honesty line that a score is the model's opinion, not fate. **Elasticity flywheel**: the price simulator's default prior is MEASURED (lifetime ceased products over base), labeled a floor not a price response, always overridable. **Cross-tenant priors** (`GET /ai/v1/elasticityBenchmark`): each tenant's baseline computed inside its own context, only the unnamed distribution leaves, and below the k-anonymity floor of three the benchmark refuses to exist. Suites `prospect_sim_test` , `chaos_twin_test` , `reg_rehearsal_test` , `elasticity_benchmark_test` .

The bill as a document — suite #46

`GET /customerBill/{id}/document.pdf` (owner-scoped); `POST /customerBill/{id}/resend` emails it to the address on file. Delivery preference per customer: `paper` | `einvoice` | `digital` (`POST /individual/{id}/billDelivery`) — the customer picks the channel; the tenant keeps the plumbing.

E-invoice formats are rows (console → Bill formats)

CODE	STANDARD	SYNTAX
<code>ehf</code>	EHF Billing 3.0 (Norway; carries the KID)	UBL 2.1
<code>peppol</code>	Peppol BIS Billing 3.0	UBL 2.1
<code>cii</code>	EN 16931	UN/CEFACT CII
<code>aunz</code>	A-NZ Peppol BIS 3.0	UBL 2.1
<code>edifact</code>	UN/EDIFACT INVOIC D96A	segments
<code>facturx</code>	Factur-X / ZUGFeRD	PDF + embedded CII

Adding a country is an INSERT (the suite added XRechnung through the form). The tenant's `BILL_DISTRIBUTION_FORMAT` picks a row by code; the renderer follows the row.

Delivery, status, and the money coming home — suites #46/#47

Deliveries (console tab): the outbox-backed ledger — every bill's trip, attempts, the buyer's Peppol Invoice Response (accepted / rejected with their words / paid), one-click Retry on failures. **Remittance**: the bank posts camt.054, Nets OCR or BAI2 to `POST /bank/v1/remittance` (per-tenant `BANK_TOKEN`); a matched KID settles the bill through the card path's own guarantee; everything unclear parks on **Unapplied cash** (console tab) with its reason. Money is fail-closed: never guessed, never dropped.

Dunning (console → Dunning)

The collections worklist: overdue installments, broken plans, what is still owed. The full **statutory collections ladder** — case state machine, country statutory pack, holds, cure, the Collections desk — is chapter [42](#).

6 • Partner channels — suites #48/#51

Being a dealer is a row. Sign a chain: `POST /dealer/v1/agreement` (staff) with the org, commission per activation, and — for a chain with its own POS — the `OAuth2 clientId` that speaks for them. Clerks are org membership; foreign dealers see 404-shaped nothing.

Retail (the CSP + external-retail model — think Elkjøp/Power)

Counter sale: sell against the customer's email; the order carries the dealer stamp. **Starter kits:** mint batches (code + SIM + attribution in the box); the customer self-activates (`POST /dealer/v1/starterKit/activate`) and the line rides the SIM from the box. **POS API:** same endpoints, machine credential, per-partner rate limits at the edge (429 + Retry-After); the chain's own phone rides the sale as `device` context — never a billable item. **Commission:** pending → earned after the angreter window → clawed back with the reason if the customer leaves inside it.

Telesales (outbound, shaped by the law)

Dial list: `GET /dealer/v1/telesales/dialList?segment=` — consented at the source (insight segments), every number DNC-washed, reserved citizens excluded *and counted*. **The call makes an OFFER** (`POST /dealer/v1/telesales/offer`): washed fail-closed (reserved refuses, unreachable register refuses, unconfigured tenant refuses); no order exists yet. **The customer confirms in writing** (`POST /telesales/v1/confirm` , signed in, code from their inbox — or, for a cold prospect, after registering with the offered email); only then the order, the activation and the commission are born. Silence expires on a clock. Every dialer call logs to TMF683.

7 • The host administrator

The tenant fleet is a file

`infra/tenants/tenants.yml` is the shared registry (mounted into all 31 services); every service live-refreshes it. NEW operators join the running fleet within one refresh interval; changes to a serving operator's seams/brand/hosts apply live; identity fields (`id` , `issuer` , key endpoints) require a restart — by design.

Minting an operator — suites #49/#50

```
# the script (~2 minutes to a billed customer):
ops/onboard-tenant.sh fjord "Fjord Mobil" da DKK "#1B6CA8"

# or the form: console → Operators → five fields → Save
# (host-tenant admins only; realm clone + registry block + seeded catalog;
# shop.<id>.localhost wears the brand the moment the gateway refreshes)
# live rebrand of a form-born operator:
PATCH /onboarding/v1/operator/{id} {"name","color","locale","currency"}
```

Seed operators (genalpha, nova) are protected from the form — their config is env.

Staff & roles (console → Staff)

Grant/revoke whole areas per operator over the tenant's own IdP (TMF672) — no Keycloak console needed for daily administration.

8 • AI configuration & governance — suites #53/#59

The individualized shop (suite #60)

GET /ai/v1/forYou — the signed-in customer's own rail (self-scoped: party = token subject, no parameter to probe). TMF680 candidates + consented interests + churn-risk loyalty flag; ONE governed FAST caption per 5-minute cache window ("Picked for you after your look at devices"). No personalization consent → zero interests, generic caption. The storefront renders it as the "Recommended for you" shelf, and the mobile app's For-you card rides the same endpoint (same cache — the second channel is free).

The control plane (suite #59)

Every LLM turn is **metered** (token counts + cost land on the AI Audit ledger — the console's AI Audit tab shows tokens, cost and outcome per turn) and **governed** per tenant: GET /ai/v1/governance shows spend this window, the ceiling, remaining headroom and the **agent-action trail** (which AI wrote which resource — e.g. the advisor's catalog.createDraftOffering). POST /ai/v1/governance/budget {"budgetMicros": 5000000, "windowHours": 720, "enabled": true} sets the ceiling and the **kill-switch**; crossing the ceiling refuses fail-closed (429, audited as refused-budget), enabled=false refuses instantly (403). No budget row = unlimited-and-enabled — governance is opt-in per tenant. Cost is estimated (chars/4 tokens × per-model rate, bss.ai.prices.<model> in micros per 1K, default bss.ai.default-price-per-1k-micros); exact provider usage is a wired-later seam.

Per tenant, all data: ai-provider (stub — keyless CI default — openai-compatible , anthropic), ai-base-url , ai-api-key , ai-model . **Tiered routing**: the task class lives at the call site — FAST (copywriting, summaries, narratives) vs SMART (product proposals, next-best-offer; the default when unclassified). Every value resolves tier-first:

```
AI_MODEL_FAST=swift-mini           # cheap model for volume work
AI_MODEL_SMART=frontier-x          # careful model for judgment
AI_PROVIDER_SMART=anthropic        # tiers go down to the PROVIDER:
AI_BASE_URL_SMART=https://api...   # a local endpoint for volume,
AI_API_KEY_SMART=...               # a frontier API for judgment — at once
```

9 • The seam catalog

Every external system is a per-tenant seam: configure it and the feature lights up; leave it blank and the feature degrades honestly (fail-open for enrichment, fail-closed for money, identity and consent).

SEAM	TENANT FIELDS (ENV)	BEHAVIOR WHEN BLANK
Email provider (ESP)	DELIVERY_PROVIDER, ESP_URL, ESP_API_KEY, ESP_FROM	in-app inbox only
Bill distribution partner	BILL_DISTRIBUTION_{PROVIDER,URL,TOKEN,FORMAT,CHANNEL}	bills stay in-app/PDF
Bank (remittance)	BANK_TOKEN	webhook refuses (money: fail-closed)
Do-not-call register	DNC_URL, DNC_TOKEN	outbound telesales refuses (opt-IN by configuring)
Market-intelligence feed	MARKET_FEED_URL, MARKET_FEED_TOKEN	advisor's market findings absent; own-data findings stand
LLM	AI_* (§8)	keyless stub answers
PIM (product imagery)	PIM_BASE_URL	internal TMF667 store
Content store (deployment-level)	CONTENT_STORE = in-row s3 azure-blob (+ S3_* / AZURE_*)	bytes in the document row (dev-simple)
Embeddings (deployment-level)	bss.knowledge.embeddings = stub openai-compatible (+ embeddings-url/key/model)	deterministic keyless stub — the semantic net still works, with curated synonyms instead of a model
Web analytics (GA4)	ANALYTICS_*	internal insight only
Social platform	SOCIAL_API_URL, SOCIAL_ACCESS_TOKEN	no audience push / lead import

Per-tenant variants ride suffixes: `_NOVA`, `_FJORD`, ... matching the placeholders in `tenants.yml`.

10 • Environment reference (dev clocks)

VARIABLE	MEANING	DEV / PROD DEFAULT
BSS_BILLING_DUNNING_TICK_MS / _GRACE_MS	collections sweep / grace	5s & 20s / 60s & 7d
BSS_BILLING_DISTRIBUTION_TICK_MS / _RETRY_SECONDS	delivery relay / backoff base	3s & 4s / 30s & 60s
BSS_SOM_COMMISSION_WINDOW_MS / _TICK_MS	angrerett window / harden tick	15s & 5s / 14d & 1h
BSS_SOM_TELESALES_EXPIRY_MS / _TICK_MS	offer expiry / sweep	15s & 5s / 48h & 1h
BSS_TENANTS_REFRESH_MS	live registry refresh	15s
BSS_GATEWAY_PARTNER_RATE_CAPACITY / _WINDOW_MS	per-partner rate limit (dealer surface)	10 per 2s / 60 per 60s
GLOBAL_RATE_CAPACITY / GLOBAL_RATE_WINDOW_MS	the wide ring — a ceiling on EVERY path, per subject/client/IP	1200 per 60s

11 • Extension API reference (beyond the TMF standard surfaces)

ENDPOINT	WHO	DOES
GET /customerBill/{id}/document.pdf	owner/staff	the bill as a PDF
POST /customerBill/{id}/resend	owner/staff	email the invoice to the address on file
POST /customerBill/{id}/dispute · /installmentPlan	owner/agent	contest / split a bill
GET/PATCH/POST /billFormatProfile[//{code}]	read: billing · write: billing:admin	e-invoice profiles as rows
GET /billDistribution · POST /billDistribution/{id}/retry	billing:admin	the delivery ledger
POST /bank/v1/remittance	the bank (X-Bank-Token)	camt.054 / OCR / BAI2 in
POST /distribution/v1/response	the partner (X-Distribution-Token)	Peppol Invoice Response onto the ledger
GET /remittance/unapplied	billing:admin	unapplied cash worklist
POST /individual/{id}/billingCycle · /billDelivery	self/staff	payday alignment · delivery preference
/dealer/v1/* (agreement, kits, sell, commission, orders/{id}, telesales/*)	staff / dealer / partner POS	the whole dealer & telesales channel
POST /telesales/v1/confirm	the customer, signed in	the written yes that births the order
GET /advisor/v1/findings · POST /advisor/v1/adopt	catalog:write	product advisor · adopt-as-draft
GET/POST /onboarding/v1/operator · PATCH .../{id}	host roles:admin	mint / live-mutate an operator
GET /app/tenant-config.json	public, by Host	the white-label manifest

12 • The mock integrations (dev stand-ins)

CONTAINER	PORT	STANDS IN FOR	CHAOS / INSPECTION
mock-esp	8121	SendGrid-shaped email provider	GET /mails , webhook receipts
mock-social	8122	Meta-shaped audiences/leads	—
mock-distribution	8124	Peppol access point / print house (validates EHF/EDIFACT/Factur-X!)	GET /invoices , POST /outage , POST /invoices/{billNo}/respond
mock-dnc	8125	reservation register	numbers ending 9999 are reserved; POST /outage
mock-market	8126	tariff-comparison feed	GET /offers
minio	9000	S3-protocol object store (AWS/R2/on-prem)	—
azurite	10000	Azure Blob Storage (Microsoft's emulator)	—
mock-llm	8127	OpenAI- and Anthropic-dialect LLM	names the model in answers; GET /requests
mock-efaktura	8147	e-invoice rail (consumer alias lookup + request-for-payment)	POST /alias/lookup · /users , GET /rfp ; ch. 46
mock-digipost	8148	digital mailbox (letters addressed by national id)	GET /letters , POST /recipients ; ch. 46
mock-avtalegiro	8149	direct-debit rail (mandates, claims, OCR settlement)	GET /mandateFile · /claims · /settlementFile ; ch. 46

13 • Production operations — suites #56/#57

Ticks that never double-fire

Every mutating scheduled job — dunning, bill delivery, campaign journeys, churn sweeps, porting cutover, commission hardening, order resume, telesales expiry, cart abandonment — claims a row-lease (`tick_lock` table in the service's own database) before it runs. Scale any service to N replicas and each tick still fires once; a crashed holder's lease expires on its own. Inspect a service's leases with `psql -d billing -c "SELECT * FROM tick_lock"`. Suite #56 proves it by running TWO billing replicas and counting exactly one delivered bill at the receiving partner.

The billing run — partitioned, resumable, and on a ledger

Each account's bill commits in its *own* transaction; a failed account is recorded and skipped, never the run. A crashed run RESUMES on the next trigger — every bill already cut is its own checkpoint — and every run is a row: `GET /tmf-api/customerBillManagement/v4/billingRun` lists recent runs (status running/completed/superseded, accounts, bills, failures, last error). One run speaks at a time: a trigger during a live run answers `{"busy": true}` (the lease heartbeats per account, so a crashed run frees it within ~2 minutes), and the database enforces one bill per account per period with a unique index — constraint, not convention. Suite #57 kills billing mid-run and proves exactly-one-bill-per-customer after the resume. `BSS_BILLING_RUN_ACCOUNT_DELAY_MS` (default 0) is the dev pacing knob the suite uses to hold a run open.

Load numbers and the smoke SLO

`node ops/load/loadtest.js --seconds 15 --workers 10` — plain-Node closed-loop driver, three scenarios (anonymous catalog, storefront page, authenticated bill reads), reporting req/s and p50/p95/p99. LIFT THE WIDE RING FIRST (`GLOBAL_RATE_CAPACITY=1000000` `docker compose up -d gateway`) or the PO ceiling will throttle the harness — that is it working. Baselines and their caveats: [docs/perf-baselines.md](#).

Alerts

`infra/prometheus/alert-rules.yml` : ServiceDown (2 min unscraped), OutboxPublishFailures (events queuing, not lost), GatewayServerErrors (5xx > 5%). The whole fleet is scraped (32 targets). View at `:9090/alerts` ; a deployment routes them to a pager via Alertmanager.

Backup and the restore drill

`ops/backup.sh` dumps every database and role to `backups/` (git-ignored, newest 14 kept). `ops/restore-drill.sh` restores the newest dump into a throwaway container, verifies databases and rows, optionally proves a sentinel (`--expect <db> "<sql>"`), and removes the container. Run the drill on a schedule — an untested backup is a hope, not a backup.

Rate limits, two rings — buckets in Redis

The dealer surface keeps strict per-partner buckets; every other path passes a generous fleet-wide ceiling (per subject for people, per client for machines, per IP for anonymous), answering `429 + Retry-After` when tripped. Dials in §10. The buckets live in Redis (`BSS_GATEWAY_REDIS_URL` ; blank = in-memory): replicas share ONE exact ceiling, a restarted gateway still refuses inside the same window, and an unreachable Redis fails open — fairness, never authorization.

The live k8s soak (on demand) — local, AWS, and Azure

The Helm chart has run on three live clusters: local k3s, AWS EKS, and Azure AKS — each with billing at 2 replicas holding ONE set of tick leases, requests served through the in-cluster gateway. The cloud runs are one command each: `ops/k8s-soak/eks-run.sh up|smoke|down` and `ops/k8s-soak/aks-run.sh up|smoke|down` (compose must stop first — one laptop, two worlds). Both wrap Terraform + registry push + managed-Postgres + Helm + smoke + teardown. The cloud-by-cloud differences (all absorbed by seams — the app never changed) are tabulated in [architecture.md §5](#); the receipts and the live-run truths each cloud taught are in [k8s-soak-plan.md](#). Smoke: `ops/k8s-soak/smoke.js` against port-forwarded gateway/keycloak.

Managed-Postgres note: point `config.dbHost` at the managed server and set `local.postgres.enabled=false` . Some tiers cap connections low (Azure B1ms $\approx$ 50) — the chart now holds only one idle connection per pool; raise the server's `max_connections` for headroom. Managed Postgres may also gate extensions (Azure needs `pg_trgm/vector` allow-listed) and demand TLS (add `sslmode=require` to the datasource URL, or relax on the server for a soak).

The secret gate

`ops/install-hooks.sh` wires `ops/scan-secrets.sh` as a pre-commit hook: any staged diff carrying a real-secret shape (Anthropic/OpenAI/AWS/GitHub/Slack keys, private-key blocks) refuses to commit. Injectable values are listed in `.env.example` ; what a real deployment must add beyond this stack — external secrets, TLS in transit, managed HA Postgres/Kafka — is the runbook, [docs/hardening.md](#).

14 · Privacy & GDPR operations — suite #58

The data passport (right of access)

GET /privacy/v1/export — a customer exports THEMSELVES with their own sign-in (no role needed; the fan-out rides their token to every service). The DPO — any user with `roles:admin` — adds `?partyId=` to export another person. The JSON names its categories and also the legal-hold categories it cannot delete.

Erasure (right to be forgotten)

POST /privacy/v1/erase {"partyId": "..."} (DPO only). Refuses 409 while the person holds ACTIVE services — terminate first. Deletes interactions, messages, marketing, carts, tickets, appointments, behavioural data; anonymizes the profile in place; disables and scrubs the login at the IdP. Bills/payments/orders/usage/agreements are retained with their legal basis named in the report. A partial fan-out answers `"status": "partial - re-run required"` — re-run until `completed`. GET /privacy/v1/erasure lists the audit trail.

Retention clocks

BSS_RETENTION_INTERACTIONS_SECONDS (party-interaction) and BSS_RETENTION_TICKETS_SECONDS (trouble-ticket, settled tickets only) — 0 = keep forever (default). Production sets day-scale values (e.g. 63072000 = 2 years); BSS_RETENTION_SWEEP_MS paces the sweep. TickGuard-guarded: replicas never double-sweep.

PCI & lawful intercept, honestly

Card data never enters the BSS — the vault stores `pspToken/brand/lastFour/expiry` (SAQ-A-shaped; a QSA attests the rest). Lawful intercept: the BSS half is warrant-gated subscriber disclosure — shaped, not built; see [docs/privacy.md](https://docs.genalpha.com/privacy.md).

15 · The agent channel (agentic commerce) — suite #64

Being shopped by AI agents is opt-in. Every operator carries `agent-commerce: off | discovery | full` in the tenant registry — `off` (dark to agents, 404), `discovery` (the product feed answers; agent checkout refused with 403 — findable, checkout stays human), `full` (delegated checkout too). Newborn operators are born `off`. Flip it on the operator form (console → Operators → Agent commerce) or by env (`AGENT_COMMERCE`, `AGENT_COMMERCE_{ID}`); the gateway enforces it live — no restart.

The surface (ACP, spec 2026-04-17)

Feed: GET /acp/product_feed — public projection of the active TMF620 catalog (unconditioned prices only, one-time preferred; unpriced offerings are skipped). **Checkout:** POST /acp/checkout_sessions with `{items: [{id,quantity}]}`, then GET/POST /acp/checkout_sessions/{id}, POST .../{id}/complete (needs `Authorization` + `Idempotency-Key` + `payment_data.token`), POST .../{id}/cancel. A session rides a real TMF663 cart; complete charges the payment token and places a TMF622 order marked `category=agenticCommerce` with the buyer as relatedParty. A replayed Idempotency-Key returns the *same* order; a fresh key on a completed session is 409.

Delegated checkout (the trust model)

Agents never hold a machine identity. The shopper's token is exchanged (OAuth token exchange, Keycloak ≥26.2) through the confidential `bss-agent` client — `fullScopeAllowed` off, scope-mapped to exactly `catalog:read` `ordering:read/write` `payment:read/write` — so the agent's credential can order and pay and nothing else. The shopper's client must carry an audience mapper naming `bss-agent` : the realm grants the delegation path explicitly, or the exchange is refused. Suite #64 decodes both tokens to prove the downscoping.

MCP tools (Claude-class agents)

`integrations/mcp-server` wears the same surface as five retail tools: `search_offerings` , `get_offering` (with "also bought"), `start_checkout` , `update_checkout` , `complete_checkout` . Shopper env: `BSS_SHOPPER_USERNAME/PASSWORD` ; exchange client: `BSS_AGENT_CLIENT_ID/SECRET` .

Honest scope: the merchant surface is implemented and proven. Appearing *inside* ChatGPT/Perplexity additionally requires the operator's business onboarding with those platforms (feed registration, PSP delegated-payment agreement) — an operator act, not a code gap. The endpoints they register are the ones suite #64 exercises.

16 • The digital workforce — suite #65

The badge is the opt-in. An AI worker is hired like staff: mint a login, grant the `digital-worker` bundle (console → Staff, or `integrations/hermes-worker/hire-worker.sh`). The grant sheds the walk-in customer defaults — a worker is staff. Revoke the badge to fire; the tenant's AI kill-switch stops the whole workforce mid-shift.

The job

`GET /ai/v1/workforce/tasks` — the open queue, derived LIVE from real backlogs (unassigned TMF621 tickets, unapplied cash). `POST .../tasks/{id}/claim` (15-min lease, `BSS_WORKFORCE_LEASE_SECONDS`), `.../complete` (VERIFIED: a ticket task completes only when the ticket is actually resolved), `.../escalate` (a reason required — escalations are counted, not punished). `GET .../ledger` is the shift record, including the worker's self-reported model usage (labeled: its own word — the worker brings its own LLM).

Approvals (the T3 gate)

Refunds, cease, erasure: the worker FILES (`POST .../approvals` — method + `/tmf-api/` path + body + reason); nothing executes. A human approves on the Workforce tab and the stored action runs under *the approver's own token*; refusing keeps its receipt too. Filing is `workforce:use` ; deciding is `ai:use` — a worker can never approve its own ask.

The scoreboard — and the controls

Console → **Workforce**: completed / escalated / deflection, average handle time, **reopen rate** (completed tickets re-checked live — the honesty metric), the crew by name and observed type, approvals queue, shift ledger, self-reported cost (labeled), and human-minutes saved — an estimate that SAYS so, computed from operator-set baselines (`BSS_WORKFORCE_BASELINE_MINUTES_TICKET/-CASH`). The tab also holds the operator's two levers: **Crew ceiling** (governance `maxWorkers` ; a new worker past it is refused at claim time — surge staffing never outgrows it) and **Hire a worker** (pick a job — care / back-office / generalist — the badge is minted and the credentials shown ONCE with the matching job card to run; activating is the runtime's act, firing is the Staff-tab revoke).

Surge staffing

The staffing signal — `staffing` in the KPIs, Prometheus gauges (`bss_workforce_backlog_depth / _active_workers / _surge` , per tenant) and `WorkforceSurgeEvent` on the boundary — tells an autoscaler when the crew is behind. Scale with KEDA (`integrations/hermes-worker/k8s-surge-example.yaml`) or the dev watcher (`surge.sh`); the ceiling caps whatever the scaler does. The BSS signals and refuses; it never spawns worker processes itself.

The Hermes package (day 1)

`integrations/hermes-worker/` : `hire-worker.sh` (the badge, shown once), `config.snippet.yaml` (Hermes `~/hermes/config.yaml` MCP block with the tool whitelist), and `SKILL.md` job cards (care-triage every 15 min, cash-matching daily). Runtime-neutral by construction — any MCP-speaking agent hires into the same job, badge and audit.

17 • The wrapped legacy estate (overlay) — suite #67

A legacy BSS is just another seam. Three per-tenant registry fields — `legacy-catalog-base-url` , `legacy-fulfilment-base-url` , `legacy-ticket-base-url` (empty = native mode) — wrap an existing estate: its catalog federates read-through into the native list and the ACP feed (legacy-prefixed, cached, fail-soft); orders carrying `legacy-` items hand off to the legacy work-order queue; the digital workforce works the legacy incident backlog age-stamped, with completion VERIFIED against the legacy system's own state. Never two writers: legacy masters its data, genalpha masters engagement. Also: the workforce package's closed loop — dashboard Hire starts a running worker container via the controller (`--profile workforce`); the worker's LLM is live registry config (`worker-ai-*`) rolled onto by the controller.

18 • Generative discoverability (GEO) — suite #68

AI crawlers read HTML only — and the operator holds the switch. Per-tenant `ai-visibility`: `open` | `search-only` | `dark` executes at `robots.txt`: `open` welcomes all crawlers (+ `llms.txt`, honestly labeled speculative); `search-only` keeps classic search and disallows AI answer/training bots (GPTBot, ClaudeBot, PerplexityBot, Google-Extended, CCBot...); `dark` disallows everyone. Newborn operators default `search-only` .

The gateway DUAL-SERVES `/shop/offering/{id}` by User-Agent: crawlers get bot-readable HTML with `schema.org Product/Offer JSON-LD` rendered live from TMF620 (no authoring, no syncing — suite #68 proves bot price == catalog price); humans get the SPA unchanged. `/robots.txt` , `/sitemap.xml` , `/llms.txt` generate per tenant from the same source. Follow-ups, named: insight ai-answer referrer tagging; knowledge-base FAQ pages as a public help center.

19 • The capability map

What's here, what isn't — TM Forum business capabilities (verb-first, implementation-free) mapped to tech capabilities (ODA functional blocks / TMF Open APIs) and the component carrying each, across eTOM-style domains. Every  cites its proof suite; every  carries its caveat (mock-default PSP, OCS facade, NRDB shaped-not-connected); the  column names what a deployment brings from elsewhere (production OCS, tax engine, ERP/GL,

roaming settlement, fraud/RA, FSM, real OSS) — each behind an existing seam or the overlay pattern (suite #67). The full map: <docs/capability-map.md> · executive view (filterable, GENERATED from the md so the faces cannot drift): <capability-map.html>.

20 · Loyalty — reward & retain — suite #69

The program is data; membership is a choice. Component #34 (`loyalty` , TMF658, :8114) is a points LEDGER wired between engines that already exist. The program — earn rate, gigabyte price, voucher percent, tier thresholds, expiry clock (0 = never) — is marketer-editable data (campaign:write). Customers OPT IN on the storefront My page or the app's My-points card; nobody is enrolled by default. Every point movement is an append-only journal row carrying its cause: `bill:<id>` , `redeem:...>` , `expiry:<date>` , `adjust:<reason>` (goodwill credits REQUIRE a reason).

Earn — a settled bill earns at the program's rate, idempotent per bill (the journal's unique cause index), members only. **Burn** — three ways, each landing on an existing engine and verified there, never on loyalty's own word: points → gigabytes on THIS month's meter via the usage allowance-boost mechanic; points → a single-use discount voucher minted as a REAL TMF671 promotion (unique code, valid at the same `checkPromotion` door as any campaign code, second redemption 409s); tiers → PRICING, because `loyaltyTier` rides the billing pricing context and one console-authored rule ("Loyalty tier benefit" preset) prices gold differently — the pricing engine needed zero new code.

Govern — points are a liability: the TickGuard-guarded expiry sweep journals every expired point (FIFO arithmetic), and `GET .../liability` returns the outstanding-points number a finance team books.

Retention wiring — `LoyaltyTierChangedEvent` rides the outbox onto `bss.loyalty.events` , which the campaign engine hears like any business event: the console's campaign form now opens with RECIPES ("Churn save: a loyalty offer to at-risk customers" on the churn scorer's alert; "Loyalty: congratulate a tier change") that prefill trigger + message and stay fully editable. Suite #69 proves all nine legs end to end, including a real tier flip landing a real TMF681 message, and paula's points card composed onto her app Home.

21 · Revenue export — the subledger — suite #70

The BSS is the subledger; the ERP is the ledger. Component #35 (`revenue` , house API `/revenue/v1` , :8107 — no TM Forum Open API covers the GL, deliberately) turns billing and payment events into balanced double-entry journal postings: invoice issued → AR debit / revenue credits per rate line (discounts as contra-revenue); cash books ONCE, at payment capture, whichever door the money came through (card, CSR, bank giro via remittance); refunds book the reverse. Balance is a save-time INVARIANT and every posting is idempotent per source ref — at-least-once delivery can never double-book.

The chart is data — `GET/POST /revenue/v1/accountMapping` (billing:admin): finance's own codes and names per posting key; booked lines keep the snapshot they were born with, so a remap never rewrites history. **The export** — `GET /revenue/v1/journalExport?date=` (CSV, one row per line) is the file a period-close import ingests; `journalEntry` serves the same data as JSON; the console's read-only **Journal** tab shows entries with their lines.

The tie-out — `GET /revenue/v1/reconciliation?date=` : per-account totals, billed vs cash, every entry balance-checked live, and the loyalty points liability as a labeled CONTROL number (no currency valuation is invented — pricing a point is P2 config). `POST /revenue/v1/backfill` onboards pre-arc bills idempotently. P2 dials, all data: a VAT percent on the `tax` key splits postings into net revenue + tax payable (gross-minus-nets — rounding can never unbalance); a per-point value on `loyalty:liability` turns the points control number into a daily booked accrual; `POST /periodClose` makes the export FINAL (postings into a closed period 409); dispute credits on unpaid bills

book contra-revenue automatically; `?format=sap|netsuite` emits ERP-shaped layouts. Rev-rec (ASC 606/IFRS 15) stays in the ERP's engine; this feed is its clean input. See [docs/revenue-export-plan.md](#).

22 • Credit notes — suite #71

The wrong invoice is never edited — a numbered credit note reverses it. `POST /customerBill/{id}/creditNote {amount?, reason}` (billing:admin — agents open disputes, admins decide money; the 403 wall is suite-proven): mints `CN-NNNNNN` from billing's gapless per-tenant `document_sequence` (row-locked — the bookkeeping rules demand an unbroken series), references the original bill number, and **REQUIRES** a reason. Unpaid bill → the due comes down (a negative `creditNote` line says why; zero settles). Settled bill → the money moves **BACK** through the PSP (settlement `refunded`, `refundRef` kept). Dispute resolutions with `outcome=credit` now mint their own credit note (`disputeId` carried). Customers read their own notes (`GET /creditNote`, PDF at `/creditNote/{id}/document.pdf`) — the storefront shows a `kreditnota` chip per bill; the CSR 360 has the issue button. The revenue subledger books **REDUCED** notes as contra-revenue under the document's number and books nothing extra for **REFUNDED** ones (the refund event owns that). P2 shipped: `GET /creditNote/{id}/document.xml` is the UBL `CreditNote-2` (type 381, EHF/Peppol shape, `BillingReference` to the invoice) and partner-wired tenants ship it on the **SAME** distribution ledger as bills; `lines: [{id, amount?}]` credits named rate lines (each reversal names what it reverses, capped per line); the back-office console's Disputes tab is the decide workload.

23 • Process flows & the incident agent — suite #72

Choreography runs the flows; TMF701 explains them. Component #36 (`process`, :8116, `/tmf-api/processFlowManagement/v4`): design intent as editable `processFlowSpecification`s (each task's owed event + time allowance), run-time `processFlow`s **PROJECTED** from ordering/SOM events with the per-flow cross-system timeline persisted, and **STUCK AS A STATE** — the TickGuard sweep fails a task past its allowance, out loud on `bss.process.events`. Operators recover via `PATCH .../taskFlow/{id}`; customers see their own flows (self-service order tracking); console: Process flows tab.

The agent whose memory compounds (in `intelligence`, IG1547-aligned): a failed task triggers context assembly (spec + failed step + timeline) and a **GOVERNED** diagnosis — same kill-switch/budget/meter/audit doors as every AI call; its **ONLY** write is a ticket note (Lo). Every investigation is an episodic trace with a **MANDATORY** human verdict (`POST /ai/v1/incident/{id}/verdict`). Three useful verdicts on one signature draft a runbook (governed, provenance to source traces) that stays `proposed` until approved (console: Runbooks tab; deciding — approve/reject/revoke — is `ai:admin` like contracts and budgets, `ai:use` reads the library); the next occurrence auto-diagnoses **FROM MEMORY** — no model call, audit as witness — and `GET /ai/v1/incident/stats` carries the learning curve as data. Revocation sends diagnosis back to the LLM path: the brake is first-class. L2 auto-remediation stays out of scope until per-taskFlow idempotency is earned. See [docs/process-memory-plan.md](#).

24 • The standards sweep — suites #73–76

Fulfilment (TMF700+697, component #37, :8117) — physical orders mint `shippingOrder`s (warehouse PATCHes states, tracking ref), install bookings mint `workOrder`s; **DELIVERED** + visit **COMPLETED** completes the product order machine-driven. Customers track their own deliveries; state changes are staff-only. **SLA (TMF623)** — an `sla` block on the agreement characteristic (circuit, `thresholdMinutes`, `creditAmount`, `capPerMonth`); late-resolved problems mint violations on assurance's capped ledger (`/tmf-api/slaManagement/v4/sla|slaViolation`) and billing compensates with the **PRE-AGREED** credit note, in-process, no one deciding at breach time. **Event hub**

(TMF688, component #38, :8123) — POST /hub callback + filter (billing:admin), delivery ledger with backoff retries and dead-letter. **Thin faces:** TMF724 incidents over the agent's memory (verdicts are the state machine), TMF653 serviceTests with history over diagnose, TMF639 pools + issued-number ledger with the honest generator note. See [docs/standards-completion-plan.md](#).

25 · The product configurator (TMF760) — suites #77/#78

Two anonymous task resources on product-catalog at /tmf-api/productConfigurationManagement/v5 (TMF760 has no v4), stateless — the POST computes from the catalog's own rows and answers. **Query:** POST /queryProductConfiguration with {productConfiguration:{productOffering:{id}}} returns the configuration space — fixed members, choice groups (minSelections / maxSelections /default), each option's pickers and prices with appliesWhen conditions visible. **Check:** POST /checkProductConfiguration with selectedOption:[{id}] + configurationCharacteristic:[{name,value}] validates BOTH pick bounds, characteristic values scoped to the PICKED options, and policy block rules (under bss-catalog , the catalog's machine identity; fail-open) — rejections use ordering's exact wording. An approved answer carries configurationPrice (monthly + one-time + lines; conditioned lines only on exact match; the deal engine's indicative total rides along) and an ORDER-READY productConfiguration echo with each pick on the option whose spec owns it. **Agent channel:** MCP tools get_configuration_options and configure_product (an approved answer includes a ready checkoutItem); ACP checkout items accept configuration — the cart prices from the check, refuses invalid picks before a session opens, and complete builds nested order items with product.productCharacteristic , so billing rates the configuration like any other channel's order. See [docs/tmf760-configurator-plan.md](#).

26 · Service qualification & the coverage map (TMF645) — suites #79/#80

TMF679 answers the commercial question (may this offering be sold here); TMF645 answers the technical one, on the same `qualification` service at /tmf-api/serviceQualificationManagement/v4 . The substance is the **coverage map** — operator-editable rows {technology, postcodePrefix, maxDownMbps, maxUpMbps} ; an EMPTY prefix matches everywhere (the mobile fallback), longest matching prefix wins per technology; seeded by ops/seed/seed_coverage_map.py , CRUD gated by qualification:write . **Query:** POST /queryServiceQualification with {searchCriteria:{place:{postCode}}} — every technology deliverable there, with bandwidth characteristics. **Check:** POST /checkServiceQualification — verdict per requested service (name the technology via serviceSpecification.name or a characteristic; optional minDownstreamMbps), and NEVER a bare no: a refusal carries eligibilityUnavailabilityReason AND alternateServiceProposal with the best technology the address can have. Checks persist and read back anonymously by unguessable id; the LIST (customer addresses) is back-office. **Channels:** ordering qualifies every placed item at create (same TMF679 check the storefront runs; fail-open; refusals speak the qualification's words), and the cart's address block names what the network delivers ("Fiber up to 1000 Mbit/s at your address"). See [docs/tmf645-service-qualification-plan.md](#).

27 · AI management — the control plane's face (TMF915) — suite #81

The governed-AI machinery, standards-addressable on intelligence at /tmf-api/aiManagement/v4 . **GET /aiModel** (ai:use): the models the audit ledger PROVES have served — distinct provider/model/tier pairs with the scenarios each served, plus the trained churn classifier carrying its trainingRecord (sampleCount, positives, trainedAt). **GET /aiModelContract[/{id}]** (ai:use): one contract per governed scenario — state (active | suspended |

haltedByKillSwitch), servedBy model refs, monitoring (calls, tokens, costMicros, avgLatencyMs, and per-outcome counts including refused-budget / refused-disabled / refused-contract), and the guardrail block (tenant kill-switch, budget, window). **PATCH /aiModelContract/{id}** (ai:admin — the seam, now open): {state: suspended|active, note} — the per-scenario brake, checked in the governor's gate order right after the tenant kill-switch; a suspended contract's calls 403 and land on the ledger as refused-contract . Budget-setting (POST /ai/v1/governance/budget) now also requires ai:admin — as do runbook decisions and workforce hire/fire (the worker-controller probes GET /ai/v1/governance/adminCheck and trusts the BSS's 200/403 as the verdict; the console hides ceiling+hire from non-admins, who keep the scoreboard and approvals). Ops note: accessTokenLifespan is 1800s in BOTH realm files — a Keycloak container recreate re-imports realms from files, so live-only settings (and dynamically-onboarded realms like aurora) do not survive one. See [docs/tmf915-ai-management-plan.md](https://docs.tmf915-ai-management-plan.md).

28 • Risk management — the door check (TMF696) — suite #82

The acquisition-side twin of churn, on intelligence at /tmf-api/riskManagement/v4 behind risk:assess (demo staff + the bss-ordering machine; customers 403 even on their own scores — a risk score is a credit check). **POST /partyRiskAssessment** {relatedParty:[{id}]} and **POST /productOrderRiskAssessment** (adds totalQuantity, lineCount, verifiedIdentity — supplied by the caller whose session knows it; a verified session scores 20 lower). The score is TRANSPARENT: named signals (unpaidBills, creditNotes, orderVelocity, newAccount, bulkOrder, verifiedSession) each with points + raw evidence in the body — recomputable by hand. Deliberately NOT scored: failed payments (refusals never persist) and lifetime verification (session claim only). Assessments persist and read back by id. **Enforcement is data:** ordering fetches a fresh assessment (machine identity, fail-open) into the policy context as riskScore / riskLevel ; the operator's JsonLogic rule sets the threshold — deny returns 422 POLICY_DENIED . Billing's GET /creditNote grew a relatedPartyId filter for the engine. See [docs/tmf696-risk-plan.md](https://docs.tmf696-risk-plan.md).

29 • Partnership types & geographic sites (TMF668 + TMF674) — suites #83/#84

TMF668 on the agreement service at /tmf-api/partnershipTypeManagement/v4/partnershipType (reads agreement:read, writes agreement:write): a partnership KIND is {name, description, status, roleType: [{name, description}]} — a kind without roles is refused. An agreement with agreementType: "partnership" whose characteristic.partnershipTypeId names a kind is VALIDATED at create: every engagedParty.role must be one the kind permits (refusals name the permitted list; unknown/retired kinds refuse outright). Untyped agreements are untouched. **TMF674** on geographic-address at /tmf-api/geographicSiteManagement/v4/geographicSite (reads address:read, writes address:write): a site is {name, status: planned|active|retired, relatedParty, place:{id}} where place.id MUST reference a stored TMF673 geographicAddress (validated at create, embedded on every read); filter with ?relatedPartyId= ; PATCH moves the lifecycle; a site's address is reusable on orders — entered once. See [docs/tmf668-674-thin-tail-plan.md](https://docs.tmf668-674-thin-tail-plan.md).

30 • The proof run & fleet-age runbooks — suite #85 and the receipt

The proof run: ops/run-all-suites.sh runs all suites serially with a readiness gate (IdP + gateway must answer before each suite), inter-suite pacing (a young fleet out-runs its own per-subject rate limiter), the surge controller

parked except for the suite that tests it, onboarding suites first (a recreated Keycloak forgets dynamic realms), rate-limit suites last, and one automatic retry pass with attempts recorded. The receipt renders to `docs/proof-run.html` via `docs/build-proof-run-html.py`. **The pruning runbook:** `ops/prune-dev-fleet.sh` retires active subscriptions owned by anyone who is not a named demo persona (nothing deleted — products stop billing), then seed one usage record per persona (meters derive from monthly usage). **Fleet-age laws** (from [docs/proof-run-findings.md](#)): machine list-clients page to exhaustion or filter server-side (an unfiltered list ages out of any fixed page — `GET /revenue/v1/journalEntry?sourceRef=` exists because of this); batch endpoints get batch timeouts; suites probing aged personas should mint purpose-made customers; ad-hoc suite runs park `bss-worker-controller` first. Billing runs on a worker pool (`bss.billing.run-concurrency`, default 8; crash-resume proven by suite #57 against the pool) — suite #85 asserts the aged-fleet run completes inside suite patience.

31 • The shop that knows you (personalization) — suite #86

Usage-aware upsell: the signed-in `GET /ai/v1/forYou` payload carries an `upsell` block when any meter is $\geq 80\%$ used — `{bucketName, usedPct, currentAllowance, suggestedOffering, suggestedAllowance}`, the suggestion being the NEXT rung of the allowance ladder (the usage service's `usageAllowance` rows per offering, smallest larger allowance for the same usage type; rows without an offering name get a descriptive label). No meter, a comfortable meter, or no fuller rung → no block. The Shop page renders it as `data-testid=upsell-card` above the recommended rail. **Returning-visitor rules:** the personalization rule context now carries `visits` (distinct event-days per visitor) — author `{>=: [{"var":"visits"}, 2]}` rules in Rules or via the copilot. **Full-profile grid:** the Shop grid ranks singles by the visitor's whole consented `interests[]` array in browse-weight order; a rule's `heroCategory` still floats to the top. Pricing on the grid is deliberately NOT a personalization lever. Hygiene note: killed suite runs orphan their finally-cleanup policy rules — if default experiences misbehave, list `domain=personalization` rules for leftovers. See [docs/personalization-shop-plan.md](#).

32 • The CTK campaign — twenty-five kits, the closed list

Scorecard: [docs/ctk-conformance.md](#) — twenty-five official CTKs at zero failures; TMF676/681 stay intentional partials. **The list is CLOSED:** the `tmforum-rand` org publishes no kit for the remaining faces (688/696/760/915/700/697/701/724/623) — there is nothing left to run. **Runners:** modern kits (`config.json` + `index.js`) via `ops/ctk/runctk.py`; R18-era kits (raw collection + environment) via `ops/ctk/runctk-r18.py` — both structure URLs for modern newman, bake the bearer, run with `--timeout-script 120000`. Gotchas that will bite again: pass the kit dir by **absolute path** (relative once made newman fail silently and the summary read a STALE `result.json`); some kits write `pmtest.json` BEFORE their own payload injection (`runctk.py` injects config payloads itself now); secondary host-vars (`{{Service_Ordering}}`/`tmf-api/...`) hang newman on literal-var DNS — `runctk-r18.py` resolves them to the gateway origin. **One-time kit prep** (documented in [ops/ctk/README.md](#)): TMF674's example payload must reference a STORED TMF673 address; TMF638 assumes a sandbox — seed two uniquely-named probe services, the newer in state `designed`. **Contract changes the campaign shipped, fleet-wide:** `agreement type` (= `agreementType`) and `ticket ticketType` are now MANDATORY (storefront files `support`, self-heal files `incident`); a roleless partnership kind stores but permits NOTHING at signature; `serviceTest` requires `testSpecification` (the built-in virtual spec is `diagnose`); `agreement + ticket` lists page NEWEST-FIRST. New faces: read-only TMF633 `serviceSpecification` on SOM (so inventory spec refs dereference), northbound `POST /serviceOrder` (external orders recorded `acknowledged`), `POST /serviceProblem` (declared problems), and v3 task dialects for TMF645/679 that delegate to the v4 coverage/serviceable-area engines. **Gateway:** SCG 4.2+ only adds `X-Forwarded-*` for `trusted-proxies` (now set to the private ranges); services build absolute Location headers from those — keep that setting when changing gateway config. **Workforce suites:**

```
workforce_runtime / agentic_workforce need bss-worker-controller UP (start for them, stop after, then  
docker rm -f wf-*).
```

33 • Bring your own CMS/DAM — reference-mode content

What: a tenant serves product imagery from its OWN headless CMS/DAM instead of the built-in TMF667 store — opt-in per tenant. Bind via `PUT /tmf-api/documentManagement/v4/contentProvider (document:write)`; no binding = hosted DAM (in-row/S3/Azure). Suite [byo_cms_test.js](#) (#89). **Model — reference, not copy:** the row stores `ref:<provider>:<assetId>`, no bytes; `GET /document/{id}/content` 302-redirects to the CMS's own URL (the browser hits THEIR CDN), with `?rendition=thumb|card|hero|orig` (shared Rendition vocab) and a fail-open placeholder when resolve fails or a webhook marked the asset gone — never a broken image, never a 500. The catalog keeps a stable `.../content` href, so swapping providers never rewrites catalog data. **Providers:** `sanity` (named adapter — Assets API upload, `cdn.sanity.io` URL reconstructed from the asset id + projectId/dataset, transform params per rendition) and `http` (the GENERIC connector, ZERO vendor code — config JSON: `uploadUrl`, `uploadMode` `multipart|raw`, `fileField`, `authHeader/authPrefix`, `assetIdPath` JSON-pointer, `resolveBase`, `renditionMode`). Sanity is common among CSPs; Strapi (MIT, self-host) rides the generic connector. **Secrets:** the upload token and webhook secret are ENV-VAR NAMES (`secretRef`, `webhookSecretRef`) — never the value; resolved from env at call time. **Freshness:** `POST /webhook/{provider}/{tenantId}` (anonymous at the door, HMAC-verified inside — Sanity's `t=,v1=` over the raw body; the shared secret IS the auth, one opaque 401 for every failure; runs in the tenant's RLS scope). `delete` → document unavailable (placeholder); `upsert` → restored + `content_version` bump that cache-busts the delivered URL. **Real Strapi proof:** `docker compose --profile strapi up -d strapi (integrations/strapi scaffolds Strapi 5 + sqlite, bootstrap grants the public role Upload)`. The suite's real-Strapi leg is health-gated on `localhost:1337` — runs when up, skips clean when down. Fast/offline mocks: `mock-sanity :8131`, `mock-strapi :8132`. **Gotchas that will bite again:** (1) the mocks are reachable from the document CONTAINER as `mock-sanity:8080 / strapi:1337`, but the suite runs on the HOST — it rewrites the redirect host to the mapped port to fetch bytes; real Sanity/Strapi have no such split. (2) Content-addressed dedup: identical bytes share ONE CMS asset, so a webhook matches EVERY document referencing it (assert `>=1`, not `==1`). (3) `config` sent as a nested JSON object must be JSON-serialised (fixed) — `Java toString()` is not JSON. (4) `PostgresMigrationTest` is `disabledWithoutDocker`, so its migration-count assertion drifts silently — bump it when you add a migration (now 6). (5) A new downstream needs BOTH the config row AND the secret-ref env present in compose.

34 • Bring your own carrier — the delivery menu

What: a tenant configures its own parcel carriers (Helthjem, Posten/Bring, PostNord) as data, and the SHOPPER picks one at checkout — the pick drives the actual booking. Operator half: suite [carrier_choice_test.js](#) (#90); shopper half: [carrier_picker_test.js](#) (#92). **Model:** `carrier_config` (RLS) — `carrier`, `displayName`, `baseUrl`, `secretRef` (env-var NAME, never the key), methods `["home", "pickupPoint"]`, `isDefault`, `postcodePrefix` rules. Edit via `PUT/DELETE /tmf-api/shippingOrderManagement/v4/carrier` (ordering:write); `GET /carrier/deliveryOptions?postcode=` is ANONYMOUS (guest checkout reads the menu); `GET /carrier/pickupPoints` searches a carrier's points. **Routing precedence** (CarrierRouter): shopper's explicit carrier → postcode rule (longest prefix wins) → tenant default → global env carrier (unbound deployments unchanged). Fail-open: any carrier error degrades to the manual warehouse flow. **The pick rides the PLACE:** checkout writes `deliveryMethod` + `carrier` (+ `pickupPointId/Name`) onto the order's delivery place — fulfilment reads it off the place when minting the shipping order; home delivery carries the chosen carrier too, not just pickup. **Storefront:** the cart renders the WHOLE menu as vertical rows (" Home delivery · Posten/Bring — delivered to your door by Posten/Bring"; pickup rows inline the

points), and the delivery block names the parcel ("📦 Ships to you: Apple iPhone 17 Pro, your SIM card") — an eSIM-only cart shows NO delivery block. **Console:** Integrations → Logistics card (list/add/remove per tenant + **Test** = a reachability probe of `baseUrl/health`, never a booking). **Gotchas that will bite again:** (1) an UNBOUND tenant falls back to one hardcoded "Helthjem" home option — "the shop only shows Helthjem" means the menu is EMPTY, not broken; re-run `ops/seed/seed_carriers_psp.py` (suites #90/#91/#94 bind then UNBIND their own providers). (2) The bundle's phone choice pre-selects a handset (min-1 choice), so delivery legitimately shows with an eSIM — the "Ships to you" line is what keeps that from reading as a bug. (3) Option tiles that WRAP read as captions of the row above — the delivery options must stay a vertical list. (4) mock ports: logistics :8128, bring :8133, postnord :8134 (same image as bring).

35 • Bring your own PSP — cards, Klarna/Vipps/PayPal, orchestration, BNPL books

What: per-tenant payment providers — card acquirers AND redirect/BNPL (Klarna, Vipps MobilePay, PayPal) — with routing rules, failover, and settlement accounting that tells the truth. Suites: [psp_klarna_test.js](#) (#91), [psp_bnpl_settlement_test.js](#) (#93), [psp_paypal_test.js](#) (#94), [install_slot_rollback_test.js](#) (#95), [psp_card_orchestration_test.js](#) (#96), [psp_bnpl_remittance_test.js](#) (#97). **Model:** `payment_provider_config` (RLS) — `provider`, `baseUrl`, `secretRef` + `webhookSecretRef` (env-var NAMES), `methods`, `isDefault`, `priority` (lower tried first) + `currencies` (JSON list, blank = any). Edit via `PUT/DELETE /tmf-api/paymentManagement/v4/paymentProvider`; `GET /payment/methods` is anonymous (the picker). NO generic connector for money — named adapters only (mock, mockbank, stripe, klarna, paypal). **Redirect flow:** `POST /payment/session` → customer approves at the provider → return leg `POST /payment/confirm` AND webhook `POST /webhook/{provider}/{tenantId}` (HMAC `t=,v1=`, forged = 401) — BOTH confirm the same `session_ref`, idempotent, so return + webhook never double-book. Capture/refund route by what authorized: redirect providers by SESSION, cards by authorization code. Capture-on-ship = the existing order-completion capture (parcel delivered → order completes → captures), now Klarna-aware. **BNPL books:** a Klarna capture debits 1100 "BNPL / provider clearing" (a RECEIVABLE — Klarna pays later), never 1000 cash; PayPal settles at capture = cash; the payout clears it: `POST /revenue/v1/remittance` (billing:admin; provider+reference+amount; idempotent by payout ref) books DEBIT 1000 / CREDIT 1100; reconciliation reports `bnplReceivableTotal` = NET outstanding. **Orchestration:** redirect sessions fail over to another configured redirect provider when the chosen one is unreachable (response carries `failedOverFrom` — the confirm leg follows who SERVED); card charges run the routed pool (currency+priority) and fail over ONLY on connect-level errors (`ConnectException/UnknownHostException`) — a DECLINE never retries (the card said no), an ambiguous read-timeout never retries (it may have charged); one idempotency key rides every attempt; the payment records the serving provider. **Storefront:** Card + one button per redirect method from `/payment/methods`; the session is opened FIRST and the SERVED provider + session id stashed (`bss.shop.redirectpay`) so a failover still confirms correctly on return. **Console:** Payment card — providers with prio/currencies on the line, routing fields in the form, **Test** = reachability probe of `baseUrl/health`, never a payment. **Install-slot rollback** (#95): order placed, appointment 409s (slot lost) → the checkout CANCELS the order (cancel voids the card + releases stock) and says "pick another slot — not placed, not charged." **Gotchas that will bite again:** (1) real Klarna/Vipps/PayPal need merchant sandbox credentials — config only (baseUrl + secret-refs), no code change; until then mock-klarna :8135 / mock-paypal :8136 (compose sets `PUBLIC_BASE` so the approve URL is browser-reachable — a relative redirect dies at the browser). (2) payment migrations now NINE — `PostgresMigrationTest` asserts the count, bump it with every migration. (3) klarna's config lists methods `["card","klarna"]`, so it appears among card candidates but is filtered out (it has no card adapter) — harmless, by design. (4) suites #91/#94 UNBIND their providers on exit — re-seed the demo menu after.

36 · Bundle decomposition — the several things it is

What: a triple-play order (Fiber + TV + Mobile + a chosen phone + optional add-on) decomposes into **per-component leaf items** that fulfil on independent clocks; **TV reliesOn Internet**; a **physical-SIM line stays reserved** until the SIM lands; a broadband speed tier **upgrades/downgrades in place** with no rebuild. Suite [bundle_decomposition_test.js](#). **Decompose:** `ProductOrderService.expandBundles()` mints a leaf child per component stamped with `category + componentType` (`internet/tv/mobile/device`); `linkComponentDependencies()` adds a TMF622 `orderItemRelationship: reliesOn` (TV→internet). **Billing stays byte-identical:** fixed components are flagged `decomposedComponent:true` and skipped by `collectItems / collectItemMaps`, so provision, policy and serviceability are unchanged and the bundle's own prices still bill THROUGH the bundle (proven +0 charges before/after). **SOM:** `OrchestrationService` provisions LEAVES only, fulfilment is category-aware (only mobile draws MSISDN/SIM), and deferral is place-aware — `deferred = hasPlace`, and a `reliesOn` dependent is held only while its predecessor is actually placed for install. **Lifecycle:** a fiber line's `downloadSpeed` is a configurable characteristic with an allowed set (100/300/500/1000, `seed_lifecycle_characteristics.py`); an upgrade is TMF622 `action=modify` on the same product, validated by `validateCharacteristicChange` (off-menu value refused by the allowed set; numeric downgrade refused while a commitment is live); `renameModifiedServices` emits a `ServiceAttributeValueChangeEvent` to re-provision. **Storefront:** `Orders.jsx` renders an Internet/TV/Mobile family tree with honest per-component status ("Activates once your broadband is live," "Number reserved — activates when your SIM arrives," "N of M ready"); the handset nests under Mobile. **Gotchas that will bite again:** (1) the `decomposedComponent` flag is what keeps a bundle from double-charging — a fixed leaf that loses it bills TWICE. (2) Deferral keys off PLACE: forcing internet deferred with no place breaks a no-place bundle (e.g. a plan-change modify) — the rule is `deferred=hasPlace`, dependent held only if the predecessor is placed.

37 · Wholesale / open-access — selling on a wire you don't own

What: a new product line — sell retail broadband on top of a third party's fibre (open access), split by access layer **L2 VULA / L3 activated bitstream**, ordered and settled operator-to-operator over **MEF LSO Sonata**. The platform plays BOTH roles: **access seeker** (we are the retail ISP buying access) and **access provider** (a sibling tenant is the fibre owner selling it) — meeting **cross-tenant**. Suites [wholesale_open_access_test.js](#) (seeker) + [wholesale_provider_test.js](#) (provider). Portal: **partner-console at /partner** (`PARTNER_URL :8138`) — check an address (which owners serve it, at which layer), browse the L2/L3 access catalogue, place a wholesale order, read what you owe. **Parties/catalogue:** `seed_wholesale_partners.py` mints owners as Organization parties (a fibre wholesaler at L3, a bitstream provider at L2 — invented names) with TMF668 `partnershipType` + TMF651 agreements; `seed_wholesale_access_products.py` puts the L2/L3 SKUs in a "Wholesale access" category auto-excluded from the shop grid. **Coverage:** `CoverageMap` gains `accessOwner + accessLayer`; `ServiceQualificationService.accessOptions()` answers `queryAccessOptions` (anonymous). **Seeker order:** `WholesaleAccessOrder` (RLS); `WholesaleAccessClient` → `MockWholesaleAccessClient` or `SonataWholesaleAccessClient` (`@ConditionalOnProperty bss.wholesale.oss=sonata`, `async` — the owner calls back `POST .../wholesaleAccessOrder/notification`); `OrchestrationService.provisionWholesaleAccess / activateWholesaleAccess`. **Settlement + COGS:** `wholesaleSettlement` reports owed-per-owner + margin; revenue's `WholesaleEventListener` books the upstream cost (DR 5100 wholesale-COGS / CR 2100 AP-wholesale). **Provider side:** `ProviderAccessOrder` (RLS) + `WholesaleProviderService` (`acceptOrder` → `@Scheduled` `activationTick` → `notifyRetailer` → `providerSettlement`, `L3=22/L2=15`); `WholesaleProviderController` exposes `POST /mefApi/serviceOrdering/v1/serviceOrder` (anonymous) + provider views; the gateway routes `/mefApi/**`; the seeker's `SonataWholesaleAccessClient` maps an owner → its tenant (e.g. the NovaFibre owner → the `nova` tenant) and calls it with that tenant's `X-Tenant-Id` —

the one deliberate cross-tenant path, through the gateway, never around RLS. **Config:** `bss.wholesale.oss=sonata` (compose default, `WHOLESALE_OSS`). **Gotchas that will bite again:** (1) `mock-sonata` is INTERNAL-only (no host ports — 8134/8135 belong to `postnord/klarna`). (2) a new downstream client needs BOTH an `application.yml` prop AND a compose env var (a missing base-URL fails open with a WARN). (3) cross-tenant needs the tenant HOST resolvable — `curl --resolve` in tests; the owner tenant's hosts live in the `MOUNTED infra/tenants/tenants.yml`, never the gateway's own `yml`. (4) unqualified Java refs can compile stale — always confirm `mvn BUILD_OK` before trusting a deploy.

38 · Browse-path caching — the shop on a surge

What: on a campaign day (Black Friday) thousands of anonymous prospects load the SAME catalog pages; this serves them from a gateway edge cache so the surge never reaches the catalog JVM or Postgres. Suite [browse_cache_test.js](#).

Two layers: (1) the catalog emits the contract — `CacheControlFilter` sets `Cache-Control: public, max-age=${bss.catalog.browse-cache.ttl-seconds:60}` + `Vary: X-Tenant-Id` on token-absent GETs of the catalog base path, and `no-store` the instant an `Authorization` header is present (dial `bss.catalog.browse-cache.enabled`). (2) the gateway caches at the edge — `Spring Cloud Gateway LocalResponseCache` (needs BOTH `caffeine` AND `spring-boot-starter-cache` on the classpath), filter `LocalResponseCache=60s,50MB` on the `product-catalog` route only. The `TenantHostFilter` stamps `X-Tenant-Id`, the cache keys by it (via `Vary`), so `genalpha`'s catalogue is never served to `nova`. **Deliberately NOT cached:** `productStock` ("only N left"), `/insight/experience`, `/ai/**`, and the cart — all per-visitor or volatile. **Gotchas that will bite again — this one nearly shipped a bug:** (1) enabling `filter.local-response-cache.enabled` ALSO turns on the GLOBAL response-cache filter (its `global-filter.local-response-cache.enabled` flag is `matchIfMissing=true`), which caches EVERY authenticated route — carts, orders, bills — stale for the 5-minute default TTL. The symptom was a cart that rendered EMPTY (the empty-cart snapshot cached before the add). **FIX:**

`spring.cloud.gateway.server.webflux.global-filter.local-response-cache.enabled: false` — and it MUST be the `server.webflux` namespace (SCG 4.3), because the legacy `spring.cloud.gateway.*` alias has no migration for that key. (2) the gateway jar is HOST-BUILT (Dockerfile copies `target/*.jar`) — run `mvn package` FIRST (`JAVA_HOME=/opt/homebrew/opt/openjdk@17`) then `docker compose build gateway`; a bare build silently repackages the stale jar. (3) verify with `curl -D-`: a cart GET must show `no-store` (not `max-age=299`); an anonymous catalog GET shows `public,max-age=60`.

39 · The fixed wholesale console — the owner's authoring desk

What: the seller side of open-access fibre, authored from the console as data instead of seed scripts. An operator holding `wholesale:admin` gets a **Wholesale** desk in the admin console (`/console`) with five panes. Suite [fixed_wholesale_console_test.js](#) (#91). **Role:** `wholesale:admin` — a realm role (both realms, file `infra+helm` + `live`), held by `demo`. It gates coverage writes (qualification), service-catalog writes (`product-catalog`), and the settlement reads (SOM) — each accepts it *alongside* the existing service role, so a wholesale-only operator needs nothing else.

Panes: (1) **Access owners** — a form that onboards an owner as TMF632 Organization + TMF668 partnershipType "Wholesale access" (`accessProvider/accessSeeker` roles) + a TMF651 partnership agreement, sequential POSTs. (2) **Access products** — publishes an L2/L3 SKU: a recurring price + a `productSpecification` (chars `accessOwner / accessLayer / downloadSpeed` + a **serviceSpecification ref to a CFS**) + an offering in the shop-excluded `Wholesale access` category; the CFS dropdown reads the service catalog. (3) **Service specs (CFS/RFS)** — the new **TMF633 Service Catalog**: a customer-facing CFS (access, with layer/speed characteristics) relies on a resource-facing RFS (the owner's bitstream). (4) **Coverage** — the FC1 pane, paints footprint by `owner×layer×postcode-prefix`. (5) **Settlement** — read-only over `wholesaleProviderSettlement` (what retailers owe

you) + `wholesaleSettlement` (what you owe owners) with CSV export. **TMF633 model:** `service_specification` table (product-catalog, V14 + V15 RLS) at `/tmf-api/serviceCatalogManagement/v4/serviceSpecification` (GET public; write = catalog:write OR wholesale:admin); `product_specification.service_specification` is the SID product→service link. The gateway routes `serviceCatalogManagement/**` to the catalog (peeled off the SOM standards bundle — SOM's derived read-only facade was gateway-only and depended on by nothing). **Gotchas that will bite again:** (1) the console's generic `selectControl` needs `{label,value}` option OBJECTS, not bare strings — string options render as `undefined` (bit FC1's `accessLayer`; the API-only test missed it, the browser suite caught it). (2) owners/products are multi-object creates, so they are CUSTOM console panes (sequential POSTs), not generic-engine tabs. (3) a new gateway path prefix (`serviceCatalogManagement`) needs its own route — the SOM bundle had claimed it. (4) the settlement pane needs the SOM reads to accept `wholesale:admin` (done) or a pure-wholesale operator 403s.

40 • Mobile wholesale / MVNE — usage-metered settlement

What: the mobile sibling of open-access fibre. As a light **MVNO** the platform owes a host MNO for the traffic its subscribers burn; a **second rating pass** re-rates the same CDRs at a per-unit wholesale rate card into a per-period ledger — usage-metered, vs fibre's flat per-line. Suite [mobile_wholesale_test.js](#) (#92); seed `ops/seed/seed_mobile_wholesale.py` (host MNO party + TMF668/TMF651 agreement + rate card + IMSI + Light-MVNO tier). **Design:** the MVNO IS a tenant, so the pass runs IN-tenant over its own CDRs — the fibre SEEKER settlement, usage-metered. The provider (host-side AR across tenants) is a deliberate boundary. **usage service** (V11 table, V12 RLS): `wholesale_rate_card` (per usage-type per-unit rate), `wholesale_usage_ledger` (unique on tenant+period+type), `imsi_range`. `WholesaleUsageService.rateWholesale(periodStart,periodEnd)` sums `usage_records` by type × the rate card → ledger; **idempotent** (one row per period+type, so re-running never double-books and revenue never double-posts). It **never flips usage_record status**, so retail rating is untouched. **Endpoints** (under the usage `BASE_PATH`, accept `wholesale:admin` alongside `usage:read/write`): POST `/rateWholesale`, GET `/mobileWholesaleSettlement` (statement + a **reconciliation**: rated units vs live CDRs — a late CDR is flagged, revenue assurance **and the loop now CLOSES ITSELF**: a scheduled per-tenant sweep (`WholesaleRerateLoop`, window-bounded — an old settled period is a dispute, not a silent mutation) re-rates the drifted row, stamps `rerateCount/lastReratedAt` and emits `WholesaleUsageReratedEvent` with the delta; revenue books the adjustment idempotently per (ledger id, rerate #), negative deltas reverse), GET/POST `/wholesaleRateCard`, GET/POST `/imsiRange`, GET `/wholesaleUsageLedger`. **Books** (revenue): a `WholesaleUsageEventListener` on `bss.usage.events` books `postMobileWholesaleCogs` — DR mobile-wholesale:cogs (5110) / CR mobile-wholesale:payable (2110), amount off the event, idempotent by `sourceRef=mobile-wholesale:<ledger-id>`. **Console:** a **Mobile wholesale** pane on the Wholesale desk (settlement with the reconciliation badge, rate card, IMSI ranges, a period picker + "Rate this period"). **Gotchas that will bite again:** (1) a suite that rates a shared period picks up OTHER runs' CDRs — isolate by unique usage-type names per run (the suite does). (2) revenue's `journalEntry` detail returns lines under `lines`, not `journalLine`. (3) the tenant-wide CDR query (`findByTenantIdAndUsageDateBetween`) is new on `UsageRecordRepository` — the pass rates the whole MVNO, not one subscriber.

W-M7 — the PROVIDER face (suite [mobile_wholesale_provider_test.js](#), #93): the OTHER MVNO type — one that runs its **own BSS**, which we host and bill wholesale. So mobile now supports both types, like fibre. `provider_rate_card` (V13/V14) is what the host charges an MVNO per unit; a **per-MVNO row overrides the default** (`mvnoPartyId=null`) — the **SLA/tier lever**: a premium MVNO pays more than a budget one for the same traffic. `ProviderWholesaleService.recordProviderUsage` takes the network's per-MVNO usage feed → rates it at that MVNO's card → `provider_usage_ledger` (idempotent, one row per mvno+period+type). **Endpoints:** POST `/mobileWholesaleProviderUsage`, GET `/mobileWholesaleProviderSettlement` (the host's consolidated per-MVNO book = AR), GET `/mobileWholesaleStatement?mvnoPartyId=` (one MVNO's own view — the machine face its

external BSS reconciles against), `GET/POST /providerRateCard`. **Books** (revenue, mirror of the seeker's COGS): `postMobileWholesaleRevenue` DR 1210 receivable / CR 4020 mobile-wholesale revenue, idempotent by `sourceRef=mobile-wholesale-rev:<id>`; the same usage-topic listener handles the `ProviderWholesaleRatedEvent`. Seed adds two external MVNOs (Aurora premium, Fjord default). **Console:** a **Mobile — host (provider)** pane on the Wholesale desk enters per-MVNO rate cards (MVNO dropdown from the party service; blank = default) and shows the host's per-MVNO settlement + total wholesale revenue. **The provider-face closed loop:** the provider ledger has no CDR store of its own — the mediation feed IS the source — so late CDRs arrive as a corrected RE-REPORT of the same (mvno, period, type). An unchanged repeat stays a free no-op; a changed one inside the shared correction window re-rates the row in place, stamps `rerateCount/lastReratedAt` and emits `ProviderWholesaleReratedEvent` with the delta — revenue books exactly that delta (DR 1210 / CR 4020, reversed when negative), idempotent per (ledger id, rerate #). Outside the window the correction is REFUSED: a settled old period is a dispute, not a silent rewrite. **Boundary:** external-MVNO auth is gated by `wholesale:admin` for the demo — a real deployment issues each MVNO a scoped machine credential for its own statement.

36 • The honest game — referrals, streaks, right-plan, community

What: engagement mechanics with RECEIPTS — every mechanic is measurable, guarded and honest by construction. **Member-get-member** (campaign svc): a customer's stable code (`GET /referral/myCode`), one redemption per joiner, never self; the reward pays in DATA on both meters when the joiner's FIRST order completes (a referral is a customer, not a click); a velocity guard HOLDS farm-shaped payouts as a worklist; a reward arriving before the joiner's meter exists is PARKED (usage svc) and lands on their first GB record — never lost. **Streaks:** a bill settled by the bank's remittance extends `streak_bill_paid` as a CDP trait (deduped per bill, milestone events for journeys, audience-targetable). **The Right-Plan Guarantee** (`POST /ai/v1/fairPlay/sweep`): customers using under 30% of their allowance get the cheaper SAME-CATEGORY plan with 1.5× headroom — read-only, the journey only tells them. **Community Grid:** area goals with a PUBLIC score (a percentage, never a person) that unlock at target; **Klubbdugnad** ties a code to a local club org and tallies the season (`GET /referral/clubs`) — and the tally is MONEY, not a scoreboard: each rewarded club-linked conversion publishes `ClubShareAccruedEvent` (amount/currency from `bss.campaign.club-share-amount/-currency`) and revenue books DR 6150 community sponsorship / CR 2150 payable-to-clubs, idempotent per conversion — a balance the operator OWES. **Channel:** `GET /dealer/v1/leaderboard` ranks stores off the commission ledger. **Config, not code:** the weekly perk drop is a journey on a segment minting partner codes; a level tier is an audience over traits (points, streaks, tenure). **Gotchas:** (1) the reward rail requires a data bucket — that is WHY parked rewards exist; (2) rate-vs-amount lessons apply to any reward math near prorated bills; (3) suggestion candidates must share the current plan's CATEGORY or a bundle masquerades as a downgrade. **Suites:** `referral_test`, `g2_honest_game_test`, `g3_community_test`.

41 • Extending the provider seams — bind, wire, or build

Who this is for: an operator or a fork bringing the BSS to a new country/market and asking two questions — *how do I switch on what already ships* (Helthjem, Posten/Bring, PostNord, Klarna, PayPal...) and *how do I add a provider you never built* (DHL, PostNL, a national registry)? There are **three tiers**, cheapest first, and the same doctrine everywhere: **geography is config, Norway is the built-in reference set** (see <docs/freg-address-plan.md>).

Tier 1 — BIND a shipped provider (config, no code, no fork). Every shipped adapter is dormant until a tenant binds it with a config row: `PUT` the row (idempotent upsert), or use the console's Integrations cards (Logistics / Payment). `secretRef` is an **env-var NAME, never a key** — going live is pointing `baseUrl` at the real API and setting one environment variable on the service:

```
# bind Posten/Bring for this tenant (ordering:write token)
curl -X PUT $API/tmf-api/shippingOrderManagement/v4/carrier \
-H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" -d '{
  "carrier": "bring", "displayName": "Posten/Bring",
  "baseUrl": "https://api.bring.com/booking",      # real API = go-live
  "secretRef": "BRING_API_KEY",                  # name of the env var, never the key
  "methods": ["home", "pickupPoint"], "isDefault": false }'
```

```
# payments are the same shape on the payment API
curl -X PUT $API/tmf-api/paymentManagement/v4/paymentProvider ... '{
  "provider": "klarna", "baseUrl": "...", "secretRef": "KLARNA_API_KEY",
  "webhookSecretRef": "KLARNA_WEBHOOK_SECRET", "methods": ["klarna"], "priority": 10 }'
```

Tier 2 — WIRE an unshipped carrier through the generic connector (still config only). Fulfilment ships an `HttpCarrierAdapter` where *config is the adapter*: any carrier whose API takes a JSON booking and returns a tracking reference is mapped with a path + JSON pointers — proven by suite [carrier_generic_test.js](#) (#101), which points it at the Bring-shaped mock, a wire it has zero code for, and books a real order:

```
curl -X PUT $API/tmf-api/shippingOrderManagement/v4/carrier ... '{
  "carrier": "http", "displayName": "PostNL",
  "baseUrl": "https://api.postnl.example", "secretRef": "POSTNL_API_KEY",
  "methods": ["home","pickupPoint"],
  "config": {
    "bookPath":      "/v2/shipments",          # POST target for a booking
    "trackingPointer": "/shipment/reference",    # JSON pointers map the reply
    "shipmentPointer": "/shipment/id",
    "urlPointer":     "/shipment/trackingUrl",
    "labelPointer":   "/shipment/labelRef",
    "pickupPath":     "/v2/locations?postcode={postcode}",
    "pointsPointer":  "/locations" } }'
```

The safety boundary is deliberate: **money never rides the generic seam** — a failed carrier booking degrades to the manual warehouse flow, so carriers are safe to generic-connect; PSPs are named adapters only. National registries (Freg & friends) are also excluded from Tier 2: those calls carry national-ID PII and a per-country legal basis, so they get named adapters too.

Tier 3 — BUILD a first-class adapter (one class; auto-discovered). `CarrierRegistry` 's constructor takes `List<CarrierAdapter>` — Spring injects **every bean implementing the interface**, keyed by `name()`. A fork adds DHL by adding one file; no registry edit, no core change, near-zero merge conflict on upstream pulls:


```

@Component                                // discovery IS the registration
public class DhlCarrierAdapter implements CarrierAdapter {
    public String name() { return "dhl"; }    // the key carrier_config binds to

    public LogisticsClient.Booking book(CarrierConfig cfg,
        LogisticsClient.Booking req, DeliveryChoice delivery) {
        // POST {cfg.baseUrl}/shipments, auth from System.getenv(cfg.getSecretRef()),
        // honour delivery.method()/pickupPointId(), map DHL's reply into the
        // normalized Booking (consignment ref, tracking URL, display name).
    }

    @Override public List<Map<String, Object>> pickupPoints(CarrierConfig cfg, String postcode) {
        // optional – omit and the carrier simply offers no pickup method
    }
}

```

Then bind it exactly like Tier 1 ("carrier": "dhl"). The repo convention for doing it properly: ship integrations/mock-dhl (copy the mock-bring skeleton) so the adapter is provable offline, and an e2e suite that books through it — PostnordCarrierAdapter + mock-postnord + suite #90 is the worked example to copy. The same one-class pattern extends every provider seam: PSP adapters (payment), CMS/DAM providers (content, ch. 33), AI providers (intelligence, ch. 8), and the planned RegistryAdapter (country-keyed national registries; freg ships as the NO reference). **Not an SDK** — deliberately: the seam interfaces are 2–3 methods on classes that live in the service they extend, and the mocks + suites are the contract test. If adapters ever need to live OUT of tree (plugin jars a fork maintains privately), the move is publishing the interfaces as a tiny bss-provider-spi artifact — the shape is already right for it; nothing else would change. **Gotchas:** (1) name() collisions win silently in the registry map — prefix fork-private adapters ("acme-dhl"). (2) Suites #90/#91/#94/#101 bind then UNBIND providers — re-run ops/seed/seed_carriers_psp.py after (ch. 34's gotcha, it will bite here too). (3) The console Test button probes baseUrl/health — give your mock a /health or the card shows red while bookings work.

42 • Collections — the statutory ladder

What: the path from a missed payment to a restricted, suspended, or cured customer — shaped by national debt-collection and telecom law, not by a retry loop. Lives in billing (the case + policy + money) and service-orchestration (the enforcement primitives). Suite [collections_test.js](#). **The case:** one collectionCase per financial account (it aggregates ALL overdue bills — rung actions fire once per account, never once per invoice), state machine CURRENT → REMINDED → WARNED → RESTRICTED → SUSPENDED with orthogonal holds — PROMISE_TO_PAY (pauses the ladder; a broken promise resumes it), DISPUTE_HOLD (freezes only the disputed amount — the rest still ages), HARDSHIP_HOLD (manual) — and CURED reachable from any state the moment the actionable balance drops under threshold. **The policy and its floor:** dunningPolicy per tenant (ordered steps: offset days, template, enforcement, fee), sitting on a **country statutory pack it cannot undercut** — the Norway pack pins the 14-day reminder-fee gate, the kr 38 fee cap, at most two fee-bearing reminders, the one-month enforcement-notice clock and the 250 minimum actionable floor. A policy below the floor is refused with 400 at write time; the console renders the floor read-only — the law's numbers are never form fields. **Enforcement:** RESTRICTED rides a new SOM primitive (POST .../service/{id}/restrict) — a barring profile lighter than suspend: outgoing barred, data throttled, the line stays active and **emergency calls stay reachable** (the whitelist is part of the profile, asserted by the suite). SUSPENDED is the existing suspend with reason nonpayment, and **billing skips MRC accrual for the duration** — the statutory rule that no subscription charges bill through a nonpayment block. Cure auto-resumes within the sweep and posts a reconnection fee (tenant config, contractual — zeroable). Fees post as rate lines through the existing AR machinery, gated by the pack. **API** (billing base path, billing:admin writes): GET /collectionCase/{id} , POST

.../{id}/promiseToPay · /hold · /release · /writeOff , POST /collectionSweep (the tick, TickGuard-locked), GET/POST/PATCH /dunningPolicy . **Faces:** the CSR console's **Collections** desk (case worklist by state, holds, promise entry, write-off, the policy list with its statutory block worded read-only) and the storefront Bills page — an outstanding banner with pay-now and the customer's own promise-to-pay. Every rung speaks to the customer through the notification templates; where the bill-distribution seam is bound (ch. 46) the legally required letters ride it, channel recorded. **Honest edges:** late-interest accrual is not booked (rarely charged consumer-side; a named follow-up) and the hand-off to an external collections agency is a deliberate gap — the case history is the file you would send.

43 · Device commerce — financing, trade-in, withdrawal

What: the device lifecycle as first-class BSS state — a new service (:8145, /tmf-api/deviceCommerce/v1 ; TMF-styled, since no TMF device-financing API exists). Suite [device commerce test.js](#). **The agreement:** deviceAgreement = party + subscription + device (IMEI) + terms (principal, term, monthly, upgrade-eligibility rule, residual value) under a **financing model** — and the model is the port, not the product: OPERATOR_BOOK (internal driver — the operator carries the receivable, billing's instalment machinery underneath), THIRD_PARTY_LOAN (mock-bank driver — the financier owns title and receivable, pays out upfront; an upgrade fetches GET .../earlySettlementQuote and settles it) and BNPL (originates through the payment rail's BNPL adapter — origination is verified against the payment session, settlement delegates to the provider; never an operator write-off). **Total cost of ownership is a required, displayed field** on every offer face — monthly × term never hides the total. **Trade-in:** tradeInValuation (IMEI + condition answers → instant estimate from the tenant-editable tradeInResidual table) walks quoted → accepted → in-transit → graded → settled ; a grading delta is money the customer must accept — a downward revision **waits for their yes** before settling, an upward one refunds through the PSP path. A blacklisted IMEI quotes exactly zero: lost/stolen raises a deviceFlag and pushes the IMEI through the blacklist adapter seam (mock driver now; a real EIR/registry feed is deployment config, not code). **Upgrade/swap:** eligibility on the agreement (GET .../upgradeEligibility) → trade-in quote → model-specific settlement (operator-book writes off the remainder against the graded value; loan/BNPL pays the early-settlement quote) → POST .../{id}/swap , idempotent. **Withdrawal:** withdrawalCase — the statutory 14-day clock starts at the **parcel-delivery event** (the same event that activates a physical SIM), the refund includes standard shipping, and the subsidy postings reverse. **The books (revenue):** the IFRS 15 subsidy subledger, event-driven like every other posting family — operator-book activation books the subsidy as a **contract asset** (DR 1250 / CR 4030), unwound monthly; an ETF recovers the unearned subsidy against the remaining asset; a swap writes off against trade-in inventory; financed programs book full recognition at payout plus a residual-value-guarantee accrual. All idempotent per source ref. **Faces:** shop checkout carries the trade-in widget (IMEI + condition → live cart credit) and the financing chooser with per-model monthly/total; the storefront **My Devices** page shows each agreement's financing label, paid-share bar and upgrade date; the CSR console's **Device desk** (device:read) is the agreement worklist + the residual table. **Honest edges:** the monthly instalment tick driven by the billing run is a named follow-up (POST .../recordInstallment records the movement today; the per-cycle billing hook lands next), and open-banking affordability signals are a shaped, unbuilt port.

44 · Base migration — moving the installed base

What: the engine that sunsets a legacy plan, reprices a cohort, or migrates an age-banded/promo plan — a new service (:8146, /tmf-api/baseMigration/v1) that turns the price simulator into the front half of an execute-something pipeline. Suite [base migration test.js](#). **The plan:** migrationPlan = matrix rows {sourceOfferingId → targetOfferingId, deltaClass: beneficial|neutral|detrimental} + eligibility filters (in-binding treatment: defer-to-expiry / exclude / migrate-with-free-exit) + a jurisdiction pack (noticeDays = 30 for the EEA, exit-right

applicability by delta class). States `draft → simulated → armed → running → done|paused`. **The rehearsal gate is hard:** `POST .../{id}/arm` without `POST .../{id}/attachSimulation` (a real price-simulation run) answers 409 — `simulate-first` is server-enforced, and the console desk mirrors it by keeping the Arm button disabled. **Per customer:** `scheduled → noticed → exit-window → order-emitted → migrated | exited | failed | rolled-back`. The engine enforces `orderDate ≥ noticeSentAt + noticeDays` as a gate, not a convention (the suite proves a noticed customer sits through ticks with no order until the clock passes), then emits **one TMF622 modify order per subscriber** — the existing proration-safe plan-change path, untouched. **Exit rights:** a detrimental delta exposes the penalty-free exit the European telecom code grants — in-binding customers included; `POST .../customer/{cid}/exit` terminates clean with the penalty-free flag for downstream billing. Promo-expiry rolls off with a courtesy notice and no exit right (disclosed at sale — the template says so honestly); age rules (`POST .../{id}/scanTriggers`) run `auto-migrate` (notice queued ahead of the birthday) or `lapse-to-grandfather` (flag only, the customer keeps the plan). **Safety:** every migration stores a pre-migration snapshot — `POST .../customer/{cid}/rollback` emits the inverse modify order; consecutive failures past `breakerThreshold` (default 3) trip the **circuit breaker** — the plan pauses, the evidence stays. `GET .../{id}/progress` is the wave monitor by state. **Faces:** the CSR console's **Migration desk** (`migration:read`) — plan list, per-plan progress, the mirrored arm gate. **Ops note:** `MIGRATION_COMPRESS_CLOCKS=true` (compose, demo/e2e only) lifts the 30-day notice floor so suites can use day-zero notices — **never set it in production; the floor is statutory**. **Honest edges:** the per-wave expected-vs-billed reconciliation against shadow billing is a named follow-up — the simulation attach and the billing run both exist; the automated tie-out between them lands next.

45 • Usage policy — pools, caps & the roaming wall

What: the policy layer on the usage meters — household sharing, spending caps and roaming limits, two of which are statutory obligations in the EEA rather than features. Suite [usage_policy_test.js](#). **Household pools:** `allowancePool` (owner = the payer account) with member subscriptions and optional per-member caps. Consumption decrements the pool through **reserve-then-commit grants per usage record** — never the whole remainder to one session — so the classic concurrent-quota race cannot over-allocate (the suite fires 12 concurrent records against 8 GB remaining and the pool grants exactly what exists); a member past the hard cap falls back to their own allowance, and pool-covered gigabytes never rate against it. **Spend meters — one primitive, three legal faces:** a `spendMeter` accumulates rated charges per subscription per cycle under a threshold policy (`GET /spendPolicy`, `PATCH /spendPolicy/{meterType}`). The three instances: (1) the customer-settable **subscription cap**; (2) the **content-services cap** — barring of jointly billed content is free and always available, the lowest selectable limit is floored by statute (a 100 cap is refused, 250 accepted), breach blocks with a notification, and minor-registered subscriptions carry mandatory barring defaults; (3) the **roaming financial limit** — present by default (≈€50 ex VAT per the EEA roaming regulation) without being asked for, warning at 80%, hard cut-off at 100%, and `POST /roamingLimit/continue` is the customer's explicit, **audited** election to keep roaming; raise/lower/opt-out per subscription. **Auto top-up:** `PUT /autoTopupPolicy` — never default-on, the consent timestamp is part of the record; one boost per breach window (a replay is a no-op), a per-cycle cap that holds, notification per purchase, and disabled means disabled. **Travel passes & zones:** passes are time-boxed, zone-filtered boosts (`POST /travelPass`); in-window zone traffic rides the pass, out-of-window traffic charges, re-rating is idempotent. The first usage record from a new zone emits a zone-entry event — the mandatory pricing information rides the notification; the pass upsell rides the campaign engine behind it. **Faces:** the storefront **Family** page shows the pool (remaining-of-total, per-member caps) and My page carries **Usage controls** (the roaming-limit card with its editable limit); the CSR 360 reads the same spend policies and pools per customer. **The deliberate absence:** app-specific zero-rating — unlawful in the EEA under the open internet rules, so no per-app classification ever enters the rating path; free-data promos are application-agnostic bonus meters. **Honest edge:** the roam-like-home fair-use monitor (4-month predominance + capped surcharges) is a named follow-up.

46 · Norway rails — registry, directory, credit, bill channels

What: the reference geography's operational rails, each a country-keyed adapter behind a generic port — mocks keep every leg provable offline; a real rail is the same adapter with a real base URL and credentials. Suite [norway_rails_test.js](#). **Registry sync** (party-account): `POST /individual/{id}/registryLink` stores the person-id link; the sync worker polls the registry's event feed (`POST /registrySync/run` to force a pass) with an idempotent cursor — a drained feed re-polls to zero — and re-fetches on address/name/death events, updating the party and re-emitting the address-verified event the CDP already listens to. **Protected addresses** (the national kode 6/7 regime) return no street: the party is flagged, the street is scrubbed and masked *everywhere* — staff consoles included — checkout completes with pickup-point delivery and no street address anywhere on the order, and every export honors the flag. Deceased flags land on the party for care handling, never a blind update. **Directory obligation:** per-party `directorySetting` — exposure `full | partial | reserved` plus the **secret number** (a mandatory free service: total suppression). `POST /directoryExport/run` produces the delta export; the suite proves a secret number and a protected party can never appear in it. Real directory-service agreements are per-tenant deployment config behind the same exporter. **Credit decisions** (ordering): a `CreditDecisionPort` (one bureau driver + a mock) feeds the existing TMF696 risk gate at checkout. A voluntary **credit freeze** answers as its own state — the order is refused `422 CREDIT_FROZEN` with a machine-readable reason the storefront turns into the prepaid path — and only the decision + timestamp are stored, never the report. **Bill distribution** (billing): an ordered channel chain per party — **e-invoice alias lookup** → **digital mailbox** → **print** — falling through as consent flags allow, with **every send recording the channel actually used** on the delivery ledger (the audit trail collections' statutory letters need; a protected party's letter carries no street). The **direct-debit loop** closes the circle: the bank's mandate file registers the authorization, each cycle emits a claim keyed by the bill's KID, and the OCR settlement file comes home through the *existing* remittance door — the same fail-closed money path as `camt.054` — settling the bill. Env: `EINVOICE_RAIL_BASE_URL`, `MAILBOX_RAIL_BASE_URL`, `DIRECTDEBIT_RAIL_BASE_URL` (compose points them at the three mocks — ch. 12: `:8147/:8148/:8149`). **Faces:** the CSR console's **Registry** page (`party:write`) drives sync runs and directory settings; the CSR 360 carries the per-customer directory block. **Honest edges:** the production eID-broker federation is a named follow-up (the BankID-class step-up gate exists from the earlier arc; a mock broker container does not yet), carrier-billing charge ingest needs a real host-operator feed (the statutory spending caps it must respect already shipped — ch. 45), and a second mailbox driver is on demand behind the same port.

genalpha-bss · one hundred and fourteen suites, twenty-five CTKs at zero failures · the build story is *The Honest Machine* (docs/book) · the TMF vocabulary is docs/book/tmf-reference.md

47 · Real-time charging — a real OCS behind the seam

What: the Online Charging System is the network's real-time balance master — the packet core asks it for quota over Diameter Gy before traffic flows. This BSS never sits in that path: the catalog *references* a rate plan that lives in the OCS (`chargingSpecId` on the plan's spec), activation *provisions* the subscriber there, TMF654 *projects* the balance and forwards top-ups, and the OCS tells the BSS when a line is running low. Suite [sigscale_ocs_test.js](#) (#123); seam doc [docs/ocs-seam.md](#).

Which OCS, per operator: each tenant block in `infra/tenants/tenants.yml` may name its own — `ocs-provider` (`http` = the generic subscriber/rate-plan REST shape of the bundled mock and vendor gateways; `sigscale` = SigScale OCS over its TM Forum APIs), `ocs-base-url`, `ocs-username`, `ocs-password`. Tenants without these use the deployment defaults (`OCS_PROVIDER`, `OCS_BASE_URL`). One fleet can charge one operator on the mock and another on a real OCS; the demo tenant Taranga runs on SigScale, the default tenant on the mock.

The bundled real OCS: SigScale OCS (open source, Apache-2.0, 3GPP Diameter Ro/Gy/Gx, RADIUS) runs as `sigscale-ocs` in the fleet — REST and GUI on `:8155` (HTTP Basic `bss / bss-secret`), Diameter on `:3868`. Its rate plans are TMF620 offerings created in its own tooling: `python3 ops/seed/seed_sigscale_ocs.py` seeds the plans the demo catalog references (RG-DATA-2 ... RG-UNL and the priority-slice twins), each a monthly recurring price whose allowance grants the data bucket and a usage price that rates overage. Change a plan there, not in the BSS.

What you will see: order a mobile plan on Taranga → in SigScale a product appears on the plan's offering with its allowance bucket, and a service whose id is the line's MSISDN; the shop's balance page shows the OCS's remaining data; a top-up in the shop lands as a bucket in the OCS; a plan change moves the line to the new offering; a vacation hold disables the identity so the network refuses data. The OCS's balance hub posts "running low" to the BSS below its configured line (`OCS_THRESHOLD_BYTES`, 2 GB by default) and the tenant's running-low journey speaks — as a **transactional** journey (a service notice), so marketing quiet hours never park it and it never spends the marketing frequency budget.

Journey category: journeys now carry `category = marketing` (default) or `transactional`. Use `transactional` for the notices a customer expects as a customer ("running low", "line suspended", "installer on the way"); `marketing` for everything that sells.

Running a real Gy session yourself: SigScale ships a Diameter test client — `docker exec bss-sigscale-ocs sh -c 'cd /home/otp && ERL_LIBS=/home/otp/lib escript lib/ocs-*/priv/bin/data_session.escript --msisdn 4741000001 --imsi 001001123456789 --raddr 127.0.0.1 --updates 2'` — and the balance falls in both SigScale and the shop.

Not here: 5G standalone converged charging (Nchf/CHF) waits for a production-grade CHF — the seam is the same, the BSS talks TM Forum to whatever charges; voice/SMS tariffs; a CGRateS adapter for VoIP-switch estates (one more adapter in the same registry when a prospect asks).

48 · Device entitlements — what the phone may use (GSMA TS.43)

What: modern phones ask the operator's *Entitlement Configuration Server* which services the subscription includes — Wi-Fi calling, VoLTE and VoNR, SMS over IP, the data plan, an eSIM for the watch, moving the eSIM to a new phone. `device-entitlement` is that server. The phone proves the SIM with EAP-AKA over HTTP (the network's own authentication, relayed to the HSS), keeps a token, and gets its answers in TS.43's own words. Suite [device entitlement test.js](#) (#124); seam doc [docs/entitlement-seam.md](#).

The plan decides: a mobile plan's specification carries `volte`, `vonr`, `vowifi`, `smsaip`, `companionEsim`, `esimTransfer` and `dataPlanType` (Metered/Unmetered). Change the plan, and every phone on it learns the new answer at its next check-in — nothing is copied anywhere. The line's state matters too: a suspended line answers "off" for everything until it is resumed; Wi-Fi calling shows "emergency address still needed" until the customer completes the little websheet the phone opens (`/ts43/flow/vowifi`). `python3 ops/seed/seed_entitlement.py` stamps the demo plans and binds the personas' lines to demo SIMs (`REALM=taranga` for Taranga).

Binding a SIM to a line happens by itself: when a mobile line activates, the orchestrator tells the entitlement server which SIM, number, party and plan belong together; a plan change and a SIM replacement follow the same way, and a suspended or terminated line loses its entitlements through the event bus. The entitlement server asks the HSS which IMSI the SIM

carries (an MVNO's SIMs are answered by its host MNO's HSS — `auc-base-url` per tenant in `tenants.yml`). `PUT /tmf-api/deviceEntitlement/v1/subscriber` remains the hook for lines that predate the server or come from another system. The console's **Device entitlements** page (Care & Ops) shows each line in words: the plan, what the phone may use, the phones that checked in, companion devices, and every request a phone made ("answered", "SIM challenge sent", "refused"). "Ask the phone to refresh" sends the TS.43 refresh notice — a push to the phone that registered one, else an SMS — through the communication component.

Moving to a new phone: when a customer transfers the eSIM through the phone's own flow (ODSA), the orchestrator blocks the old SIM, activates the new eSIM profile on the line and the old phone stops authenticating — nothing for care to do. **Clients without a SIM** (a tablet, a web sheet) are sent to the operator's sign-in (OIDC) and come back entitled. Phones that speak the older XML dialect get the same answer as XML. Beyond VoLTE, Wi-Fi calling and SMS: carrier billing (`carrierBilling`), satellite messaging (`satellite`), a private Wi-Fi identity (`privateIdentity`) and the phone's own number are answered from the same plan. **RCS** has its own door: RCS clients fetch their configuration from `/rcs/autoconfig` (GSMA RCC.14) with the same SIM proof — on a plan with RCS they get the operator's IMS access and chat/file-transfer services, on a plan without it they get "version 0" and switch RCS off. **eSIM profiles** come from the operator's SM-DP+ (the certified profile server the operator buys): the entitlement server orders and confirms them over SGP.22 ES2+ (`smdp-base-url` per tenant), the SM-DP+'s matching id is what the activation code carries, and its download notifications turn a watch active and mark a transfer installed. The fleet ships `mock-smdp` for this; with no SM-DP+ bound the server mints a stand-in code and labels it so.

Try it without a phone: the fleet ships a simulated TS.43 client — `curl -X POST localhost:8158/simulate -H 'Content-Type: application/json' -d '{"ecsUrl":"http://gateway:8080/ts43","imsi":"242050000000001","apps":["ap2003","ap2004","ap2005","ap2010"]}'` — it runs the real EAP-AKA relay (Milenage, RFC 4187) and prints the entitlement JSON the ECS returned. Add `"apps":["ap2006"]`, `"operation":"ManageSubscription"`, `"operationType":0`, `"companionTerminalId":""`, `"companionEid":""` to activate a companion eSIM and receive its SGP.22 activation code.

The honest edge: Apple and Google must point their clients at the operator's ECS and certify it — that is the operator's (or the host MNO's) relationship and takes its own time. The server is built to pass it; the real HSS/UDM goes behind each tenant's `auc-base-url`; the eSIM profile download itself belongs to the SM-DP+.

49 • The ontology — what the BSS can do, written once

What: the business model of the BSS as a machine-readable registry — the concepts it knows (Customer, Subscription, Product offering, Service, Order, Bill, Entitlement), the governed actions that may be performed on them, the capabilities of the components that execute them — served by the `ontology` component and read by people (Platform > What the BSS can do), by applications (the generated SDK) and by AI agents (the Ontology MCP). Suite [ontology_test.js](#) (#125); seam doc [docs/ontology-seam.md](#); the research and the review it answers in [docs/ontology-research.md](#).

An action is a contract: its meaning, its inputs, every condition that must hold first (in words a refusal can quote), who may perform it (the owner themselves, or a role — a customer's token acts on its own line only), the policy domain consulted, its governance (autonomy, approval, audit), the TM Forum capability that executes it, what follows, and the events it emits. *Try a dry run* on the page: enter a subscription id and a target offering id and read a verdict per condition; nothing changes. *Explain the journey* lists the whole path in steps.

Execution stays where it was: the registry places the same TMF622 order the shop would, with the caller's own token and channel; product-ordering validates and consults policy again; SOM moves the OCS rate plan and tells the entitlement server; billing prorates the month on the next run. What the registry adds is the answer before and the receipt after — under AI & Automation > Decisions, decision point `ontology.upgradeSubscription`, with every verdict as evidence.

Your own rules: an operator overlay (`ontology/tenants/<tenant>/`) may add concepts and actions and may tighten governance and wording on a core action; it can never remove a core condition. The Taranga demo adds a monthly price ceiling; the card says "extended by this operator".

Ten live actions, one retired: upgrade, pause, resume and SIM replacement on a line; cancel an order; dispute a bill; issue a credit; request, approve and hold a launch. `issueCredit` carries the maker-checker ladder in its governance: a care agent alone up to 25, filed for a finance approver above (AI & Automation > Workforce, executed with the approver's token), never above 50. A retired action stays on record, refuses past its date and names its successor. Every journey component describes itself at runtime and *Conformance* compares that with the registry; the outcome sweep judges each receipt later (retained / changed / lost); a learning contract can be written for every `ontology.*` decision point; the CSR desk's *Upgrade options* runs on the generated SDK.

The honest edge: learning contracts state intent but do not yet gate execution; six components still describe themselves by source only; the subscription-to-line link is by name, as everywhere else today.